Syrian Private University
Faculty of Artificial Intelligence Engineering
Department of Intelligent Information Systems Security Engineering

Design and Implementation of an Autonomous Context-Aware Deception Framework

Graduation Project 1 Report

Submitted in partial fulfilment of the requirements for the Bachelor's degree in Intelligent Information Systems Security Engineering

Prepared by:
Omar Khaled Abdulkafi
Moaz Hassan Al-Dawalibi

Supervised by:
Dr. Wassim Juneidi
Eng. Nagham Ashqar

Academic Year 2026/2027

Abstract

[The abstract will be completed at the end of the semester, once the results of Graduation Project 1 are available.]

الملخص

[يُستكمل الملخص في نهاية الفصل الدراشي بعد ظهور نتائج العمل.]

Table of Contents

Abstract..... i
الملخص..... ii
List of Figures..... v
List of Tables..... vi
List of Abbreviations..... vii
List of Symbols..... ix
Introduction..... 1
0.1 Preface..... 1
0.1.1 Importance of the General Field: Cybersecurity..... 1
0.1.2 Importance of the Specific Field: Intelligent Cyber Deception..... 1
0.2 Scientific Problem..... 2
0.3 Project Objective..... 3
0.3.1 The Reference Study..... 3
0.3.2 What This Project Aims to Achieve..... 3
0.4 State of the Art: Previous Studies..... 6
0.4.1 Ghost Without Shell: Measuring Non-Interactive SSH Attacks on Honeypots. .6
0.4.2 Long-Term Observations in an SSH Honeynet..... 6
0.4.3 Cloak, Honey, Trap: Proactive Defenses Against LLM Agents..... 7
0.4.4 LLMHoney: A Real-Time SSH Honeypot with LLM-Driven Responses..... 7
0.4.5 Enhancing Generalisation of Cross-Domain Intrusion Detection..... 8
0.4.6 A Microservice-Oriented Cyber Deception Platform..... 8
0.4.7 Koney: A Cyber Deception Orchestration Framework for Kubernetes..... 9
0.4.8 D3O-IIoT: Reinforcement-Learning-Driven Dynamic Deception Orchestration 9
0.4.9 All That Glitters Is Not Gold: Exposed ICS and Honeypots in the Wild..... 10
0.4.10 SHAP Happens: An Explainable IDS for Industrial IoT Networks..... 10
0.4.11 Summary of the Related Work..... 11
0.4.12 Positioning of the Project..... 12
0.5 Practical Applications..... 12
0.6 Scientific Challenges..... 13
Chapter 1: Theoretical Foundations..... 15
1.1 Introduction..... 15
1.2 Cyber Deception and Honeypots..... 15
1.2.1 Cyber Deception..... 15
1.2.2 Honeypots and Interaction Levels..... 15
1.2.3 Honeypot Fingerprinting..... 16
1.2.4 The Cowrie Honeypot and Its Measurement Artefacts..... 16
1.3 SSH, Monitoring and the Attack Surface..... 17
1.3.1 SSH Sessions and Input Lines..... 17
1.3.2 Network Security Monitoring with Zeek..... 17
1.3.3 Attack Surface Analysis..... 17

1.3.4 Transparent Traffic Redirection....18
1.4 Behavioural Features from Early Input Lines....18
1.5 Classification Models....19
1.5.1 The Classification Problem....19
1.5.2 Gradient Boosting with LightGBM....19
1.5.3 Logistic Regression and Support Vector Machines....19
1.5.4 Hard Voting....20
1.5.5 Class Imbalance and Evaluation Metrics....20
1.6 Evaluation Methodology....21
1.7 Explainability and Attacker Modelling....22
1.8 Containers and Their Isolation....23
1.9 Moving Target Defense....23
1.10 Small Language Models in Deception....24
Chapter 2: Design of the Proposed Solution....25
2.1 Introduction....25
2.2 Architecture of the Proposed Solution....25
2.2.1 Design Decisions....27
2.3 How the Solution Realises the Objective....27
2.4 Flow Charts of the Operation Sequence....29
2.4.1 Runtime Loop of the Orchestrator....29
2.4.2 Data, Training and Validation Pipeline....30
2.4.3 Life Cycle of a Decoy....32
2.4.4 The Voting Ensemble (Model E)....34
2.5 Design Parameters....35
2.6 Pseudo-Code of the Software....36
2.6.1 Classifier Programs....37
2.6.2 Graduation Project 1 Runtime Programs....42

List of Figures

Figure 2.1: Architecture of the proposed solution at the end of GP1 (the GP2 layer is dashed)..... 26
Figure 2.2: Realisation of the project objective: sub-objectives, components and verification..... 28
Figure 2.3: Flow chart of the runtime operation sequence..... 30
Figure 2.4: Flow chart of the data, training and validation pipeline (pre-registered protocol v2)..... 32
Figure 2.5: Flow chart of the decoy life cycle: (a) deployment, (b) monitoring, rotation and retirement..... 34
Figure 2.6: Model E, the hard-vote ensemble of Amendment 2: (a) decision at input line 3, (b) adoption procedure..... 35

List of Tables

Table 0.1: Summary of the reference study and related work.....11
Table 1.1: Comparison of honeypot interaction levels.....15
Table 1.2: Cowrie event types used by the framework.....16
Table 1.3: Selected values of the conn_state field in Zeek's connection log.....17
Table 1.4: Confusion matrix with Human as the positive class.....21
Table 1.5: Container isolation mechanisms and their use in the decoys.....23
Table 2.1: Flow-chart symbols.....29
Table 2.2: Design parameters and initial values.....35
Table 2.3: Software modules and their pseudo-code.....36

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
CI	Confidence Interval
COW160x4	Public data set of 160 Cowrie honeypots in four configurations
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
CV	Coefficient of Variation
DNAT	Destination Network Address Translation
DNS	Domain Name System
DT	Decision Tree
eBPF	extended Berkeley Packet Filter
FHR	False-Human Rate
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GGUF	GPT-Generated Unified Format (model file format of llama.cpp)
GP1, GP2	Graduation Project 1, Graduation Project 2
HTTP	Hypertext Transfer Protocol
ICMP	Internet Control Message Protocol
ICS	Industrial Control System
IDS	Intrusion Detection System
IIoT	Industrial Internet of Things
IMC	Internet Measurement Conference
IoT	Internet of Things
IP	Internet Protocol
JSON	JavaScript Object Notation
KYPO	Cyber range platform of Masaryk University (source of the external human data set)
LightGBM	Light Gradient Boosting Machine
LLM	Large Language Model
LOGO	Leave-One-Group-Out (validation)
LogReg	Logistic Regression
MCC	Matthews Correlation Coefficient
MDP	Markov Decision Process
MITRE	The MITRE Corporation
MTD	Moving Target Defense
NAT	Network Address Translation
PTY	Pseudo-Terminal
Q4	4-bit weight quantisation
RBF	Radial Basis Function

Abbreviation	Meaning
RF	Random Forest
SDK	Software Development Kit
SHA-256	Secure Hash Algorithm with a 256-bit digest
SHAP	SHapley Additive exPlanations
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
SSH	Secure Shell
SVM	Support Vector Machine
TCP	Transmission Control Protocol
TN	True Negative
TP	True Positive
TTY	Teletypewriter (terminal) log
userns	User namespace (Linux kernel)
VM	Virtual Machine
XML	Extensible Markup Language

List of Symbols

Symbol	Description	Unit
t_i	Arrival time of input line i	s
x_i, $\lvert x_i \rvert$	Input line i and its length	—, characters
k	Decision line: input line at which a session is classified	—
Δt_i	Gap between input lines i and $i + 1$	s
$\tilde{g}$	Logarithm of the median gap	$\log_{10}$ s
μ_Δ, σ_Δ	Mean and standard deviation of the gaps	s
CV	Coefficient of variation of the gaps	—
r_{max}	Logarithm of the highest input rate	$\log_{10}$ (characters/s)
δ_{min}	Floor on gaps	s
E, e, $\rho(e)$	Exposed services, a service and its risk tier	—
Λ	Decision policy mapping context to a deception action	—
$\mathbf{x}$, y, $\hat{y}$	Feature vector, true class and predicted class	—
N, N_c, K	Training samples, samples of class c, number of classes	—
F_m, h_m, η	Boosted model, m-th tree or ensemble member, learning rate	—
$\mathbf{w}$, b	Weight vector and bias of a linear model	—
α_i, $K(\cdot,\cdot)$	SVM coefficients and kernel function	—
M, s	Number of ensemble members and share of Human votes	—
θ, θ_{high}	Decision threshold and threshold for the richest decoy profile	—
w_c	Weight of samples of class c	—
$\mathscr{F}$, FP_f, N_f	Bot families; false positives and sessions of family f	—
F, φ_i, φ_0	Set of features, SHAP value of feature i, base value	—
X_t, s_i, p_{ij}	State at step t, the i-th state, transition probability	—
$Q(s, a)$, r, α, γ	Action value, reward, learning rate, discount factor	—
κ, B	Addresses that define a campaign bot; bootstrap resamples	—
W, T_w	Watched ports and probe de-duplication window	—, s
T_{idle}, T_{max}, P_{MTD}	Idle time and lifetime of a decoy; mean rotation period	min

Introduction

0.1 Preface

0.1.1 Importance of the General Field: Cybersecurity

Banking, healthcare, energy distribution and public administration now run on networked information systems, and a failure in any of them reaches far beyond the IT department. Cybersecurity has consequently moved from a narrow technical topic to a strategic concern. A server exposed to the Internet is probed around the clock, mostly by automated scanners and credential-guessing bots, but occasionally by skilled human operators who log in and explore the machine by hand. Long-term honeypot measurements show that this pressure is constant and that its character is changing: since 2023, sessions that quietly gather information have outnumbered openly destructive ones. Human intrusions are rarer than automated ones, yet they are usually the more damaging. Since the cost of an intrusion rises with every hour the adversary goes unnoticed, detecting attackers early and understanding what kind of attacker one is facing are among the core goals of defensive security.

Firewalls and signature-based intrusion detection systems remain essential, but they are largely reactive. They block or flag traffic that matches a known rule, and they say very little about who the attacker is, what they are after or how they behave once inside. Attack tools keep becoming more automated, and a new class of adversary has appeared: autonomous agents driven by large language models (LLMs), which can plan and execute multi-step intrusions on their own. Defenders therefore need proactive mechanisms that gather intelligence about adversaries before real assets are affected.

0.1.2 Importance of the Specific Field: Intelligent Cyber Deception

Cyber deception approaches the same problem from another direction. Rather than only hardening real systems, the defender places decoy resources in the network whose sole purpose is to be probed and attacked. Legitimate users have no reason to touch a decoy, so any interaction with it is suspicious by definition, and the resulting alerts are unusually reliable. The honeypot is the oldest and best-known form of deception. It records the commands an attacker types, the tools they download and the timing of each action, which is behavioural evidence that firewall logs cannot provide.

Attackers have adapted, however. They routinely fingerprint honeypots, try default honeypot credentials and sometimes even use honeypots as proxies for their own campaigns; at Internet scale, a sizeable share of the systems that look like exposed industrial devices turn out to be honeypots that can be recognised as such. Honeypots with fixed configurations are losing their value as a result, and research has moved towards adaptive and context-aware deception that changes its behaviour according to what it observes. Artificial intelligence is

central to this change. Machine learning can tell different types of adversaries apart, learning agents can adjust how a decoy responds, and small language models can generate believable decoy content on modest hardware. The project combines network security monitoring, deception engineering and artificial intelligence, three areas that fall within the specialisation of intelligent information systems security.

0.2 Scientific Problem

Mature tools exist for each of the three tasks involved, namely monitoring, deception and classification. Current deception deployments nevertheless suffer from five related weaknesses.

Deception is context-blind. Honeypots are normally placed according to a configuration chosen once by an administrator. That configuration ignores which services the protected host actually exposes and which ones attackers actually probe, and it stays the same when the attack surface changes.

Deception is static and therefore recognisable. A medium-interaction honeypot shows every visitor the same emulated environment and gives the same answers. Identical, repeated behaviour and inconsistent system identities are precisely what fingerprinting relies on. A static decoy also cannot raise its realism for the rare high-value adversary while staying cheap for the bulk of automated traffic.

Automated and human adversaries are not separated in real time. Most SSH traffic that reaches a honeypot comes from bots that either run a fixed script or execute nothing at all. Only a small share comes from human operators, and their sessions are by far the most informative. Separating the two groups after a session has ended is an analysis task; a decision that has to be taken while the session is still running, from its first few input lines, is a different and harder problem.

Evaluations do not transfer to deployment. Machine-learning detectors in security are often evaluated on data drawn from the same environment they were trained on, so they may learn artefacts of that environment instead of the behaviour of interest. Recent experiments show that intrusion detection models which score highly on their own data set lose much of their performance when they are tested on data from another environment.

Automated decisions are opaque. Suppose a model labels a session as human and triggers a costly response. The analyst responsible for that response needs to know why the model decided as it did, and a black-box decision is hard to trust, audit or debug.

The scientific problem of this project can therefore be stated as a single question: how can a deception framework link what it observes about its own attack surface and about the

attacker's behaviour to an adaptive, auditable and explainable response, using a classifier whose evaluation is valid beyond the laboratory, so that realistic decoys appear where and when they are most useful?

0.3 Project Objective

0.3.1 The Reference Study

The project builds on "Q-Cowrie: An adaptive honeypot to analyse attackers' behaviour" by Var Naseri, Welch, Haseeb and Mansoori, published in the *International Journal of Information Security* (Springer) in 2026. The authors start from two observations. Static honeypots answer every attacker in the same way and are therefore easy to expose. Existing adaptive honeypots, for their part, still depend on predefined rules or on learning agents that begin with no prior knowledge.

Their work has two experimental phases. In the first, two customised Cowrie honeypots were left exposed to the Internet for twenty-one days and collected roughly 410,000 attack sessions. The commands were parsed, normalised and mapped to MITRE ATT&CK techniques. From the resulting sequences the authors built a Markov Decision Process (MDP) with eight attack-lifecycle states, ranging from initial reconnaissance to persistence, and estimated its transition probabilities from the data. In the second phase, a Q-learning agent was embedded in Cowrie to create Q-Cowrie, with a learning rate $\alpha = 0.01$, a discount factor $\gamma = 0.95$ and an exploration rate $\varepsilon = 0.1$. The agent's Q-values were initialised by simulating the MDP offline. This addresses the cold-start problem, in which an agent would otherwise have to learn entirely from costly interaction with live attackers. During operation, Q-Cowrie decides how to answer each command: it can allow it, block it with a realistic error message, delay it or reply with a provocative message. The authors report that dynamic responses succeeded in 94% of cases and that adaptability and learning improved by about 80% over the baseline.

The authors are also explicit about what remains open. Skilled attackers may still fingerprint the honeypot, and the learning model's decisions are hard for a human to interpret. As future work, they propose coupling the honeypot with intrusion detection, moving to a hybrid architecture with low- and high-interaction layers, and adopting explainable artificial intelligence. They used provocative messages only as an exploratory hint for separating human attackers from automated ones; no dedicated classifier was built. The present project starts from these open points.

0.3.2 What This Project Aims to Achieve

The project designs and implements an **Autonomous Context-Aware Deception Framework**. Q-Cowrie adapts the responses of one honeypot; the proposed framework goes

a step further and adapts the deception environment as a whole. Two kinds of context drive it: the context of the defended host, obtained through automated attack surface analysis, and the context of the attacker, obtained by classifying the behaviour of each session as automated (Bot) or human (Human).

The project spans three phases. The semester phase is complete and provided the foundation: passive network monitoring with Zeek, a Cowrie SSH honeypot reached through transparent traffic redirection, an attack surface analysis module built on Nmap, and a first Bot/Human classifier trained on laboratory sessions: bot sessions generated by scripts and human sessions typed by trained human operators. The ground truth of each session is taken from its provenance (the login identity under which it was produced), independently of any feature.

At the start of Graduation Project 1, the semester classifier and its data were re-examined against current guidance on machine learning in security. The review found that part of what separated the two classes in the laboratory came from the measurement process rather than from behaviour. Cowrie de-duplicates identical terminal logs, so keystroke records were missing more often for repetitive bot sessions than for human ones; command substitutions such as `$(...)` and the echo of piped `passwd` commands were logged as extra commands; and some features, such as the delay before the first command, reflected how laboratory sessions were started rather than who typed them. A classifier can exploit such regularities without learning the behaviour of interest, a failure known as shortcut learning. The classifier was therefore rebuilt under a protocol that was written down and frozen with SHA-256 hashes before any test data were seen:

- Input lines are reconstructed from the Cowrie log after the known artefacts are removed, and each session is classified once, at a fixed input line k ($k = 3$), so that a decision is available while the session is still running.
- Only timing features of those first lines are used: the logarithm of the median gap between lines, the coefficient of variation of the gaps and the logarithm of the highest input rate. Features that depend on the laboratory set-up are excluded.
- Models are compared by leave-one-group-out validation, in which all sessions of one human operator or one bot class are held out together, and uncertainty is reported with group-level bootstrap intervals.
- Generalisation is tested on two external, single-class data sets that played no part in training: campaign bots from a public 160-honeypot Cowrie data set (COW160x4), reviewed blind for human-operated sequences, and human trainees from the KYPO cyber range.
- The deployed model, Model E, is a hard-vote ensemble of LightGBM, logistic regression and a support vector machine. It was specified in a pre-registered

amendment together with its acceptance criteria, and it is used only because it met them; otherwise the frozen single model would have been kept.

Graduation Project 1, the current phase, turns this classifier and the attack surface report into an active, adaptive response. Its objectives are the following:

- Developing a Python orchestrator that receives Cowrie and Zeek events in real time, classifies each session at input line *k*, and decides through an explicit policy Λ which decoy to deploy for which attacker.
- Building a library of dynamic decoys as Docker containers emulating SSH, web and database services. The SSH decoy runs the host's own Ubuntu release (22.04), so that its user space matches the kernel an attacker sees, and records every input line, including those typed into interactive programs; the web and database decoys run Alpine Linux.
- Hardening the decoys: user-namespace remapping, so that root inside a container is unprivileged on the host; dropped capabilities; read-only file systems; resource limits; and an isolated decoy network without address translation, with outbound connections and name resolution blocked.
- Implementing a self-morphing mechanism: when a watched service such as a database port is probed, a matching decoy starts and the attacker's next connection is steered to it. A warm standby decoy hides the start-up delay.
- Redirecting traffic transparently through Netfilter rules generated by the orchestrator, so that the attacker meets the same host as before: Cowrie and the decoys share one host identity (host name, release, SSH host keys and version string), and the account the attacker used in Cowrie is created in the decoy before the redirect.
- Applying Moving Target Defense (MTD) to the content of the decoys: files, lure credentials, accounts and the set of watched ports change over time, while the identity of the host stays fixed.
- Generating honey-files, logs and records with a small, quantised language model (Qwen2.5 or Phi-3.5-mini class, run locally through llama.cpp). Generation happens offline, before deployment, so that the model never sits on the attacker's path and cannot alter the timing of a session.
- Recording every deception decision with its reason in an SQLite database, and preserving the evidence of each decoy with SHA-256 checksums before it is removed, so that the framework's behaviour can be audited and replayed.

Graduation Project 2 will complete the framework with Markov-chain prediction of attacker intent over ATT&CK states, SHAP-based explanations of every classification decision, automated forensic reports, an interactive dashboard and red-team validation, including

signals aimed specifically at LLM-driven attackers. Taken together, the three phases answer the future directions that the reference study set out.

0.4 State of the Art: Previous Studies

This section reviews ten studies from 2025 and 2026 that bear directly on the proposed framework, in addition to the reference study presented in Section 0.3.1. They were selected to cover the decisions taken in this phase: the behaviour of attackers on SSH honeypots, the separation of attacker types, the use of small language models, the validity of evaluations, container-based and orchestrated deception, fingerprinting, and explainability. Each review summarises what the authors did and how their work connects to this project. A summary table and a short discussion of where the project stands relative to this body of work close the section.

0.4.1 Ghost Without Shell: Measuring Non-Interactive SSH Attacks on Honeypots

Valeros, Sladić and Garcia tested a widespread assumption in SSH honeypot research, namely that attackers log in, open an interactive shell and then type commands. For fifteen days they ran eleven SSH honeypots that accepted both interactive shell requests and non-interactive command execution requests, collected 177,622 authenticated sessions, and checked their results against an independent Cowrie data set from the same period. Non-interactive sessions, in which the client executes a single command through the SSH exec request, made up 99.23% of the total; interactive shells accounted for only 0.10%. The authors argue that honeypots which measure success by session length or by the number of typed commands overlook most authenticated attacks and can draw misleading conclusions about attacker behaviour.

For the Bot/Human classifier, this confirms that interactive, human-like sessions are rare, so class imbalance has to be treated as a central concern. It also explains a design decision: the classifier decides at the third input line, so it judges only sessions that reach that line, and the share of sessions it covers must be reported together with its accuracy. Finally, the decoys must answer non-interactive exec requests as convincingly as interactive shells.

0.4.2 Long-Term Observations in an SSH Honeynet

Munteanu, Suriyanarayanan, Smaragdakis, Feldmann and Fiebig studied data from a honeynet of 221 identically configured Cowrie honeypots spread over 55 countries, collected between December 2021 and August 2024. Of more than 635 million recorded sessions, they focused on the 163 million in which the attacker logged in and ran commands. They sorted the commands into 59 behavioural categories and clustered sessions by the Damerau–

Levenshtein distance between tokenised command sequences, which let them follow botnet families such as Mirai and Gafgyt over time.

Three of their findings matter here. From early 2023 onwards, sessions that merely gathered information outnumbered destructive ones, which points to more exploratory behaviour. Attackers actively fingerprinted the honeypots: after logging in with Cowrie's default username, more than 90% of these sessions disconnected immediately. A number of attackers also used the honeypots as proxies for their own campaigns. The authors conclude by calling for a new generation of honeypots built on controlled, containerised environments. These results feed directly into several design choices of the framework: default honeypot artefacts are removed, selected attackers are escalated to containerised decoys, decoy profiles change over time, and outbound traffic from decoys is blocked.

0.4.3 Cloak, Honey, Trap: Proactive Defenses Against LLM Agents

Ayzenshteyn, Weiss and Mirsky considered an attacker that is neither a classic bot nor a human: an autonomous penetration-testing agent built on a large language model. Such agents read and trust the text they encounter, including service banners, file contents and command output. The authors turned this dependence into a defence and organised it into six strategies and fifteen techniques, from content that misleads or stalls the agent to traps that reveal its presence. They implemented the techniques in a tool called CHeaT and evaluated them against agents attacking eleven capture-the-flag machines, showing that planted content can derail or expose such agents. The paper appeared at the USENIX Security Symposium 2025.

The study matters for the framework in two ways. First, it shows that the Bot/Human distinction is not exhaustive: an LLM agent issues commands at machine-like speed but with human-like variety, so timing alone may misclassify it, and the framework treats such sessions as a known limitation to be measured rather than ignored. Second, its techniques suggest a complementary signal. Because the decoys are filled with generated content, they can also carry canary items aimed at language-model agents, a layer planned for the validation in Graduation Project 2.

0.4.4 LLMHoney: A Real-Time SSH Honeypot with LLM-Driven Responses

Malhotra built an SSH honeypot that answers common commands from a virtual file system and passes unknown commands to a language model, and asked which models are small enough for this role. Thirteen models between 0.36 and 3.8 billion parameters were compared on 138 Linux commands on a laptop with 8 GB of memory, measuring response quality, how often the models invented inconsistent output, and latency. Most models needed two to four seconds per command (Qwen2.5-1.5B about 2.0 s, Phi-3 3.8B about 3.6 s), and some took ten seconds or more. The rate of invented output ranged from 5.8% for Phi-3 to

25.9% for the weakest small model. The author concludes that models of 1.5 to 3.8 billion parameters give the best balance, but that a language-model honeypot needs several times more computing power than a conventional one.

This study is the main reason for two changes to the language-model component planned in the original proposal. The proposal named TinyLlama (1.1B), an older model at the small end of this range; the study shows that the choice of model within the range matters considerably, so the framework now selects among Qwen2.5 and Phi-3.5-mini class models by a comparison defined in advance. More importantly, a per-command delay of several seconds would itself be a fingerprint and would distort the very timing features on which the classifier relies. The framework therefore uses the model only offline, to pre-generate decoy content before deployment, and never in the path of a live command.

0.4.5 Enhancing Generalisation of Cross-Domain Intrusion Detection

Amin, Shen, Ishak, Manickam and Karuppayah studied what happens when a network intrusion detection model trained on one data set is used on traffic from another. They trained deep-learning detectors on each of four public NetFlow data sets (derived from UNSW-NB15, BoT-IoT, ToN-IoT and CIC-IDS2018) and tested them on the others, after removing IP addresses from the features so that the models could not rely on them as a shortcut. Compared with the in-domain results, the cross-domain results showed a substantial loss of performance in every metric. As a remedy the authors proposed a stacked ensemble whose base learners (GRU, LSTM, DNN and MLP) are each trained on a different data set and combined by a light meta-learner; it improved the F1 score on UNSW-NB15 by more than 75% relative to the average cross-domain model, at the cost of some in-domain accuracy. The study was published open access in *Connection Science*.

The study gives the methodological justification for the way the classifier is now evaluated. A score measured on data from the training environment says little about real attackers, so the project holds out whole human operators and bot classes, freezes the model before testing, and tests it on two external data sets collected by other teams in other environments. The study also supports the use of an ensemble of different learners, as in Model E, to make the decision more robust.

0.4.6 A Microservice-Oriented Cyber Deception Platform

According to Shahzad and Saleh, most honeypot deployments are monolithic and statically configured, which limits how well they scale, adapt and provide visibility. Their platform consists of loosely coupled services: containerised honeypots, a lightweight orchestration layer driven by an application programming interface (API), and a centralised telemetry pipeline. They evaluated it in a reproducible cyber range with scripted workloads emulating reconnaissance, authentication attempts and interactive sessions. Container instantiation

took about 1.2 seconds under warm-start conditions, against tens of seconds for virtual machines. Resource usage grew roughly linearly under moderate concurrency, and the telemetry pipeline handled more than 18,000 events per minute without loss. Stress tests revealed that telemetry processing, not orchestration, was the main bottleneck.

These measurements are the main empirical reason for choosing Docker containers as the dynamic decoys of Graduation Project 1. The study's distinction between cold and warm starts also motivated the warm standby design of the framework, in which one decoy per service is kept ready so that an attacker never waits for a container to start. Finally, the finding that telemetry, not orchestration, limited the platform shows that collecting evidence from the decoys deserves as much care as the orchestration logic; this is why evidence is collected and checksummed before any decoy is removed.

0.4.7 Koney: A Cyber Deception Orchestration Framework for Kubernetes

Kahlhofer, Golinelli and Rass trace the slow uptake of deception technology mainly to operational concerns. In response, they introduced deception policy documents that describe deception techniques "as code", together with Koney, a Kubernetes operator that automates the set-up, rotation, monitoring and removal of traps in containerised applications. Koney relies on cloud-native technologies such as service meshes and extended Berkeley Packet Filter (eBPF) instrumentation to insert traps into running applications without touching their source code. The authors stress maintainability, scalability and simplicity as preconditions for adoption in practice. The work was presented at the Workshop on Active Defense and Deception, co-located with IEEE EuroS&P 2025, and the tool is open source.

Koney's declarative view of deception is close to how the orchestrator keeps its decoy catalogue as configuration and each deployment decision as a structured record. Its automatic rotation of traps is, in effect, the MTD principle that Graduation Project 1 applies to decoy profiles. Koney targets Kubernetes clusters, while this project runs on a single Docker host, but the life cycle of a decoy is the same in both: deploy, rotate, monitor and remove.

0.4.8 D3O-IIoT: Reinforcement-Learning-Driven Dynamic Deception Orchestration

Wushishi and colleagues proposed D3O-IIoT, a framework that orchestrates several defensive mechanisms at once for industrial Internet of Things networks. The defender's problem is formulated as a Markov decision process, and a deep reinforcement learning agent with a dueling deep Q-network chooses, from the observed state of the network, how to combine honeypots, MTD, fake telemetry and isolation of suspicious nodes. The work was published in *Scientific Reports* in December 2025.

D3O-IIoT supports the view that deception and MTD are most effective when they are orchestrated together and adapted to the observed situation, which is the premise of the framework. It also illustrates the alternative that this project deliberately did not take for its first version. A learned policy can adapt to situations nobody anticipated, but it is hard to audit and needs either a simulator or many interactions to train. The framework uses an explicit, deterministic policy Λ whose every decision is logged with its reason; the Markov formulation is reserved for intent prediction in Graduation Project 2, where its output informs the analyst rather than acting on its own.

0.4.9 All That Glitters Is Not Gold: Exposed ICS and Honeypots in the Wild

Mladenov, Erdődi and Smaragdakis measured how many of the industrial control systems (ICS) that appear to be exposed on the Internet are in fact honeypots. Using Internet-wide scan data and honeypot fingerprinting, they found that between 15% and 25% of the exposed ICS devices they observed were honeypots, and for some protocols the share reached 92%. Honeypots can therefore be told apart from real devices systematically and at Internet scale. This distorts measurements of the real exposure and also warns attackers away from the decoys. The study appeared at IEEE EuroS&P 2025.

The lesson for the framework is that fingerprinting is systematic and automated, not the work of a few curious attackers. A decoy must therefore be consistent with itself and with its host at every level an attacker can inspect. This is why the SSH decoy runs the host's own Ubuntu release instead of Alpine Linux: the kernel that an attacker sees through `uname` is the host's kernel, whose version string names its release, and any other user space on top of it would be an implausible combination. For the same reason Cowrie and the decoys present one identity (host name, release, SSH host keys and version string), and Cowrie's default artefacts, such as its default user and its Debian file system, are replaced by those of the decoy.

0.4.10 SHAP Happens: An Explainable IDS for Industrial IoT Networks

Loi, Canavese, Regano, Maiorca and Giacinto proposed an explainable intrusion detection system (IDS) for industrial Internet of Things (IoT) networks that pairs a Random Forest classifier with SHapley Additive exPlanations (SHAP). They evaluated it on the CIC IoT-DIAD 2024 data set, which contains more than 27 million labelled network flows. Hyperparameters were tuned to maximise the Matthews correlation coefficient (MCC), a metric chosen because it stays reliable when classes are imbalanced. The tuned model reached an MCC of 0.929 and an accuracy of 95.77% on the test set. The SHAP analysis made the model's reasoning visible: ICMP flood attacks, for instance, were detected mainly because of a very low average packet inter-arrival time and a high packet rate, which matches how these attacks work.

SHAP is planned for Graduation Project 2, and this study shows that per-alert explanations built on timing features are feasible and agree with how the detected attacks actually work. It also supports the MCC as the primary evaluation metric under class imbalance, which is the situation of the Bot and Human classes in this project. Because the framework's deployed model is a hard-vote ensemble rather than a single forest, its explanations will be computed with a model-agnostic SHAP estimator on the ensemble's vote share and, where useful, per member.

0.4.11 Summary of the Related Work

Table 0.1 lists the reference study and the ten related studies, together with the way each one relates to the project.

Table 0.1: Summary of the reference study and related work

Title	Author(s)	Publisher	Year	Relationship to the project
Q-Cowrie: An adaptive honeypot to analyse attackers' behaviour	Var Naseri, Welch, Haseeb, Mansoori	Springer (Int. J. of Information Security)	2026	Reference study: adaptive Cowrie honeypot with MDP and Q-learning; its future work defines the project's scope
Ghost without shell: Measuring non-interactive SSH attacks on honeypots	Valeros, Sladić, Garcia	arXiv preprint	2026	Most SSH sessions are non-interactive; informs class imbalance, the decision at line 3 and its coverage
Attacks come to those who wait: Long-term observations in an SSH honeynet	Munteanu, Suriyanarayanan, Smaragdakis, Feldmann, Fiebig	ACM (IMC '25)	2025	Exploratory behaviour and fingerprinting of honeypots; motivates containerised, rotating decoys with blocked egress
Cloak, honey, trap: Proactive defenses against LLM agents	Ayzenshteyn, Weiss, Mirsky	USENIX (Security '25)	2025	LLM-driven attackers as a third class; limitation of timing features and canary signals for GP2
LLMHoney: A real-time SSH honeypot with large language model-driven dynamic response generation	Malhotra	arXiv preprint	2025	Choice of a small model instead of TinyLlama; generation kept offline because of per-command latency
Enhancing generalization of cross-domain intrusion detection: a heterogeneous deep stacked ensemble approach	Amin, Shen, Ishak, Manickam, Karuppayah	Taylor & Francis (Connection Science)	2026	Justifies group-wise validation, freezing and external testing; supports an ensemble of different learners
Microservice-oriented cyber deception platform with containerized honeypots and real-time telemetry	Shahzad, Saleh	MDPI (J. of Cybersecurity and Privacy)	2026	Justifies Docker decoys and the warm standby through measured start-up latency; telemetry design
Koney: A cyber deception orchestration framework for Kubernetes	Kahlhofer, Golinelli, Rass	IEEE (EuroS&P Workshops)	2025	Deception as code and trap rotation; supports the orchestrator and MTD

Title	Author(s)	Publisher	Year	Relationship to the project
D3O-IIoT: Deep reinforcement learning-driven dynamic deception orchestration for industrial IoT security	Wushishi et al.	Springer Nature (Scientific Reports)	2025	Joint orchestration of honeypots and MTD; contrast with the auditable policy Λ; MDP view for GP2
All that glitters is not gold: Uncovering exposed ICS and honeypots in the wild	Mladenov, Erdődi, Smaragdakis	IEEE (EuroS&P)	2025	Systematic fingerprinting; motivates one consistent host identity for Cowrie and the decoys
SHAP happens: An explainable IDS for industrial IoT networks	Loi, Canavese, Regano, Maiorca, Giacinto	IEEE (RTSI 2025)	2025	SHAP explanations and MCC under imbalance; basis of the explainability layer

0.4.12 Positioning of the Project

Every building block of an intelligent deception framework has received recent attention: the long-term evolution of attacker behaviour, the dominance of non-interactive SSH sessions, new kinds of automated adversaries, small language models as deception engines, container-based infrastructure and orchestration, joint orchestration of deception and MTD, fingerprinting at Internet scale, generalisation of detection models across environments and explainable classification. The reference study contributes adaptive responses within a single honeypot.

What is missing is a combination of these pieces. None of the reviewed studies pairs automated analysis of the defender's own attack surface with an online, externally validated classification of each session as automated or human, and then uses both to decide which decoy to deploy, how to vary it and how to justify the decision. The Autonomous Context-Aware Deception Framework is designed around this combination. Monitoring and attack surface analysis supply the context, the classifier identifies the type of adversary, and the orchestrator deploys and rotates hardened, containerised decoys filled with pre-generated content. In the final phase, each of these decisions will also be explained to the analyst.

0.5 Practical Applications

The framework suits environments where remotely accessible services have to stay exposed and where early, well-characterised detection has real operational value. The main applications are the following:

- **Protection of Internet-facing servers.** Organisations that must keep SSH, web or database services reachable can divert hostile connections into decoys while legitimate users continue to reach the real services.
- **Early warning for security operations centres.** Legitimate users have no reason to touch a decoy, so a session classified as human and escalated to a rich decoy is a high-

confidence alert. It can be forwarded to analysts together with the reasons it was raised and the preserved evidence.

- **Threat intelligence collection.** Commands, keystroke-level input, timing and downloaded artefacts are stored in structured form and mapped to MITRE ATT&CK, giving a current picture of attack tools and techniques, including those of LLM-driven agents.
- **Cloud and virtualised infrastructure.** Container-based decoys are light enough to run next to cloud workloads, where attack surfaces change too often for static honeypots to keep up.
- **Education and research.** University networks and cyber ranges can use the framework to observe attacker behaviour safely and to build labelled data sets of automated and human intrusion sessions under a documented protocol.

0.6 Scientific Challenges

Building an autonomous, context-aware deception framework raises several scientific and technical challenges:

- **Fingerprinting and identity consistency.** Attackers look for default usernames, unrealistic file systems, identical responses and implausible combinations of system properties. Decoys therefore have to change over time and stay consistent with their host, without becoming unstable or impossible to reproduce.
- **Dominance of non-interactive sessions.** Most authenticated SSH sessions run a single command without ever opening a shell. A classifier that decides at the third input line can only judge sessions that reach it, and this coverage limit must be stated rather than hidden.
- **Ground truth, artefacts and shortcuts.** Human sessions are rare and hard to label. Labels must come from provenance, independently of the features, and the measurement process itself must be checked for artefacts that a model could exploit instead of behaviour.
- **Generalisation beyond the laboratory.** A model tuned on laboratory sessions may not transfer to real attackers. External data sets are single-class and come from different set-ups, so they can test the false-human rate and the human recall separately but not a complete confusion matrix.
- **Real-time constraints.** Classification, decoy selection and container start-up all have to finish before the attacker notices an unusual delay. Cold starts are slower than warm ones, which is why a standby decoy is kept ready.

- **Isolation and safety of decoys.** Containers share the host kernel, container runtimes have had escape vulnerabilities as recently as 2025, and honeypots have already been abused as proxies against third parties. Decoys have to be isolated in depth, yet the restrictions must not make them look obviously artificial.
- **Consistency and cost of generated content.** Small language models can produce inconsistent output, and generating at attack time would add a delay of seconds. Generated content must be constrained, validated against the decoy profile and prepared in advance.
- **New kinds of attackers.** LLM-driven agents do not fit the Bot/Human dichotomy. Their effect on a timing-based classifier has to be measured, and complementary signals are needed.
- **Concept drift and explainability.** Attacker behaviour changes over time, so the classifier needs periodic re-validation. Its explanations, in turn, must remain faithful to an ensemble model and understandable to analysts.

Chapter 1: Theoretical Foundations

1.1 Introduction

This chapter introduces the concepts on which the framework relies. It starts with deception and honeypots, then covers the SSH protocol, network monitoring, attack surface analysis and traffic redirection. It continues with the timing features and the classifiers that decide whether a session is automated or human, and with the methodology used to evaluate them. It ends with explainability, attacker modelling, containers, Moving Target Defense and small language models, noting for each concept where the framework uses it.

1.2 Cyber Deception and Honeypots

1.2.1 Cyber Deception

Cyber deception is a defensive strategy in which the defender deliberately introduces false resources or information in order to mislead an adversary, observe what the adversary does and delay the attack. Its strength comes from an asymmetry: a decoy has no legitimate users, so every interaction with it is suspicious by construction and its alerts carry very few false positives. Deception artefacts include **honeypots**, which imitate complete hosts or services; **decoys**, a broader term that includes services created on demand; and **honeytokens**, fake data such as credentials or documents whose use signals a compromise. The framework uses a honeypot and dynamic decoys filled with generated honeytokens.

1.2.2 Honeypots and Interaction Levels

A honeypot is an information system resource whose value lies in being probed, attacked or compromised. Honeypots are usually classified by the level of interaction they allow, which determines both how rich the collected data is and how risky the deployment becomes (Table 1.1).

Table 1.1: Comparison of honeypot interaction levels

Level	What is offered to the attacker	Data obtained	Risk and cost
Low	Emulated network stack and service banners	Connection attempts, scanned ports	Very low risk; cheap; easy to fingerprint
Medium	Emulated shell with a fake file system	Credentials, commands, timing, downloads	Low risk; moderate realism
High	Real operating system and services	Complete behaviour, real tools and malware	Highest realism; risk of abuse; costly

The framework combines the levels: a medium-interaction honeypot receives all SSH traffic, and only selected attackers are escalated to real services running in hardened containers, so

the cost and risk of realism are paid only where they are warranted. In this sense the framework is dynamic with respect to the attack surface of the host and adaptive with respect to the behaviour of each attacker.

1.2.3 Honeypot Fingerprinting

Fingerprinting is how an attacker works out that a system is a honeypot: default configuration values, missing or wrongly emulated commands, unrealistic files and answers that never vary. Low- and medium-interaction honeypots can be identified at Internet scale from small deviations in their protocol implementations, attackers routinely try default honeypot credentials and leave as soon as one is accepted, and a large share of the industrial devices that appear to be exposed on the Internet are recognisable honeypots. A decoy must therefore be consistent at every level an attacker can inspect, from banners and operating-system identity to file contents and timing.

1.2.4 The Cowrie Honeypot and Its Measurement Artefacts

Cowrie is an open-source SSH and Telnet honeypot of medium interaction, written in Python. It accepts logins according to a configurable user database, presents an emulated shell backed by a fake file system, and logs every event as a JSON object with an event identifier, a timestamp, a session identifier and the source address; the raw terminal input and output of a session can also be recorded in a TTY log. Table 1.2 lists the events the framework uses.

Table 1.2: Cowrie event types used by the framework

Event identifier	Use in the framework
`cowrie.session.connect`	Start of a session and source address
`cowrie.login.success`	Login time and provenance of the session
`cowrie.command.input`	Input lines and their timestamps, the basis of all features
`cowrie.log.closed`	Location of the session's keystroke record
`cowrie.session.closed`	End of the session

A honeypot log is a measurement, and it has artefacts that a classifier cannot tell apart from behaviour. Three of them matter here. Cowrie names each terminal log after the hash of its content and discards a new log with the same hash, so repeated bot sessions lose their own keystroke record while human sessions, which are rarely identical, keep theirs. A line containing a command substitution such as `$(...)` is logged as an extra input event a fraction of a second later. And when a script pipes text into `passwd`, the honeypot's own output can appear as an input line. The framework removes these artefacts when it reconstructs input lines, and does not use features whose presence depends on the logging process.

1.3 SSH, Monitoring and the Attack Surface

1.3.1 SSH Sessions and Input Lines

The Secure Shell (SSH) protocol provides secure remote login over TCP port 22 through three layers: transport, user authentication and connection. Inside the connection, a **shell** request starts an interactive shell on a pseudo-terminal, while an **exec** request runs a single command; most authenticated SSH sessions on the Internet are of the second kind. In an interactive shell a person types each keystroke and submits the line with Enter, whereas a script usually writes whole lines in one burst. The unit both cases share is the **input line**, the text the shell receives with its arrival time. Human input lines follow the tempo of reading, thinking and typing, with typical inter-key intervals of about a fifth of a second, while a script can deliver a long line in a few milliseconds.

1.3.2 Network Security Monitoring with Zeek

Network Security Monitoring is the passive collection and analysis of network data in order to detect and respond to intrusions. Zeek is an open-source monitor whose event engine reassembles traffic and parses protocols, and whose scripts write structured logs. Its connection log, `conn.log`, holds one record per connection with its endpoints, duration, byte counts and final state (Table 1.3). In the framework, a refused or unanswered connection to a watched port is a probe that can trigger a matching decoy. Checksum offloading on the capture interface has to be disabled, or Zeek told to ignore checksums; otherwise Zeek discards the packets whose checksums appear invalid.

Table 1.3: Selected values of the conn_state field in Zeek's connection log

State	Meaning
S0	Connection attempt seen, no reply
SF	Normal establishment and termination
REJ	Connection attempt rejected
RSTO / RSTR	Connection reset by the originator / responder

1.3.3 Attack Surface Analysis

The attack surface of a system is the set of ways through which an adversary can enter it and cause damage. At network level it is dominated by the listening services, which Nmap discovers by scanning ports and detecting service versions. For deception to be context-aware, the scan result is turned into a deployment decision. Let E be the set of exposed services, $\rho(e)$ the risk tier of service e and Λ a deployment function:

$$d(e) = \Lambda(e, \rho(e)), \quad e \in E \tag{1.1}$$

Λ returns the decoy to deploy, if any, and the redirection rule that sends the matching traffic to it. The scan is repeated periodically, and a probe against a watched service that does not exist on the host can also trigger a decoy, while a real service is never shadowed.

1.3.4 Transparent Traffic Redirection

Decisions are put into effect with Netfilter, the packet-filtering framework of the Linux kernel. A rule in the PREROUTING chain of the **nat** table rewrites the destination of the first packet of a new connection: REDIRECT changes the port on the local host, and DNAT changes the address and port, for example to a container. Connection tracking applies the same translation to the rest of the connection, so the attacker sees an ordinary session. Because a translation is fixed when a connection starts, a new DNAT rule affects only the attacker's next connections, and because capture happens before PREROUTING, Zeek still logs the original destination port.

1.4 Behavioural Features from Early Input Lines

A deception decision is useful only while the attacker is connected, so the classifier decides once, when a session reaches a fixed **decision line** k, using only its first k input lines; the pre-registered protocol selected $k = 3$ from $\{3, 4, 5\}$ before any test data were seen. Let $t_1 < t_2 < \ldots$ be the arrival times of the input lines $x_1, x_2, \ldots$ of a session. The gaps between consecutive lines are

$$\Delta t_i = t_{i+1} - t_i, \qquad i = 1, 2, \ldots, k-1 \tag{1.2}$$

The first feature describes the pace of the session through the median gap, taken on a logarithmic scale because gaps range from milliseconds to minutes:

$$\tilde{g} = \log_{10}\left(\operatorname{median}\{\Delta t_1, \ldots, \Delta t_{k-1}\}\right) \tag{1.3}$$

The second feature describes the regularity of the gaps. With their mean and standard deviation

$$\mu_\Delta = \frac{1}{k-1}\sum_{i=1}^{k-1}\Delta t_i, \qquad \sigma_\Delta = \sqrt{\frac{1}{k-1}\sum_{i=1}^{k-1}(\Delta t_i - \mu_\Delta)^2} \tag{1.4}$$

the coefficient of variation is

$$CV = \frac{\sigma_\Delta}{\mu_\Delta} \tag{1.5}$$

A script that waits exactly two or exactly twenty seconds between lines has a coefficient close to zero, while the gaps of a person vary and give a clearly positive value. The third feature is the highest input rate: the length $|x_i|$ of each line after the first divided by the gap before it, with a floor $\delta_{\min} = 1$ ms:

$$r_{\max} = \log_{10}\left(\max_{2 \le i \le k} \frac{|x_i|}{\max\left(\Delta t_{i-1},\ \delta_{\min}\right)} \right) \tag{1.6}$$

A long line that arrives a few milliseconds after the previous one could not have been typed, which makes $r_{\max}$ a strong indicator of scripted input. Several features of the semester classifier were deliberately excluded: the delay before the first command, which reflected how laboratory sessions were started and is missing in one external data set; command counts and command contents, which identify scripts rather than operators; and keystroke features whose presence depends on the logging artefacts of Section 1.2.4. A feature must describe the behaviour of interest, not the way the data were collected.

1.5 Classification Models

1.5.1 The Classification Problem

The classifier is trained on labelled sessions $(\mathbf{x}_i, y_i)$, where $\mathbf{x}_i$ is the feature vector at line k and $y_i = 1$ for Human and 0 for Bot. Human is the positive class because it is rare and triggers the most valuable response. Labels come only from provenance, the account under which each laboratory session was produced. The deployed model, Model E, combines three learners of different kinds by voting.

1.5.2 Gradient Boosting with LightGBM

Gradient boosting builds an ensemble of small decision trees sequentially, each tree correcting the errors of the ensemble built so far:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta\, h_m(\mathbf{x}), \qquad m = 1, \dots, M \tag{1.7}$$

where h_m is the m-th tree, fitted to the negative gradient of the loss, and η is the learning rate. LightGBM is an efficient implementation that grows trees leaf-wise; because this can overfit small data sets, the configuration used here limits the number of leaves, the depth and the minimum number of sessions per leaf.

1.5.3 Logistic Regression and Support Vector Machines

Logistic regression models the probability of the positive class as a logistic function of a linear score:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp\left(-(\mathbf{w}^{\top}\mathbf{x} + b)\right)} \tag{1.8}$$

Its boundary is a hyperplane, so it has little capacity to memorise peculiarities of the training data, and its errors tend to differ from those of trees. A support vector machine (SVM) separates the classes with the largest possible margin; with a kernel K its decision function is

$$f(\mathbf{x}) = \operatorname{sign}\left(\sum_{i=1}^{N} \alpha_i \, y_i \, K(\mathbf{x}_i, \mathbf{x}) + b \right) \tag{1.9}$$

where the coefficients α_i are non-zero only for the support vectors. With the radial basis function (RBF) kernel the boundary becomes a smooth non-linear surface, more flexible than the hyperplane of logistic regression and smoother than the axis-aligned, piecewise boundaries of trees. Both models are sensitive to feature scale, so every feature is standardised with the mean and standard deviation of the training fold only.

1.5.4 Hard Voting

An ensemble combines classifiers in the hope that their errors cancel out. In **hard voting**, each of the M members votes for a class and the ensemble predicts the majority:

$$\hat{y}(\mathbf{x}) = \arg\max_{c} \sum_{m=1}^{M} \mathbb{1}[h_m(\mathbf{x}) = c] \tag{1.10}$$

With three members and two classes no tie is possible. The framework also records the share of Human votes,

$$s(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} \mathbb{1}[h_m(\mathbf{x}) = \text{Human}] \ \in \left\{0, \tfrac{1}{3}, \tfrac{2}{3}, 1\right\} \quad (M = 3) \tag{1.11}$$

so that the rule $s \geq \theta$ with $\theta = 0.5$ is the majority vote and $s = 1$ marks a unanimous decision. Voting helps only if the members make different errors, which is why the members are a boosted tree model, a linear model and a kernel model.

1.5.5 Class Imbalance and Evaluation Metrics

Automated sessions far outnumber human ones, so a classifier could reach high accuracy by always predicting Bot. Each member therefore uses balanced class weights, giving every sample of class c the weight

$$w_c = \frac{N}{K \cdot N_c} \tag{1.12}$$

where N is the number of training samples, K the number of classes and N_c the number of samples of class c. Resampling with SMOTE is reported only as a comparison, and only inside training folds, because resampling before the split leaks information into the test data. The outcomes of a binary classifier form a confusion matrix (Table 1.4).

Table 1.4: Confusion matrix with Human as the positive class

Actual class	Predicted Human	Predicted Bot
Human (positive)	True positive (TP)	False negative (FN)
Bot (negative)	False positive (FP)	True negative (TN)

Accuracy is misleading under imbalance, so the primary metric is the Matthews correlation coefficient, which uses all four cells and is high only when both classes are handled well:

$$\mathrm{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \tag{1.13}$$

External test sets that contain only bots allow no full confusion matrix. The relevant quantity there is the **false-human rate** (FHR), the share of bot sessions predicted Human, also averaged over bot families $f \in \mathcal{F}$ so that no single family dominates:

$$\mathrm{FHR} = \frac{FP}{FP + TN}, \qquad \mathrm{FHR}_{\mathrm{macro}} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \frac{FP_f}{N_f} \tag{1.14}$$

On a set that contains only humans, the relevant quantity is the recall, the share of human sessions predicted Human.

1.6 Evaluation Methodology

Machine learning in security is easy to evaluate badly: sampling bias, data snooping, spurious correlations and lab-only evaluation are common, and models that score highly on their own data set often fail on data from another environment. A related risk is **shortcut learning**, in which a model exploits a cue that happens to correlate with the label, such as a logging artefact, instead of the intended behaviour. The framework uses four defences.

- **Group-wise validation.** In leave-one-group-out (LOGO) validation each human operator and each bot class forms a group that is held out as a whole, so the model is always tested on an operator it has never seen; nested LOGO repeats the model selection inside each fold.

- **Group bootstrap.** Every metric is reported with a 95% interval obtained by resampling whole groups or bot families rather than single sessions, which would treat related sessions as independent.
- **External single-class tests.** Generalisation is tested on campaign bots from the public COW160x4 data set and on human trainees from the KYPO cyber range. Recurring bot sequences are reviewed by a person before any prediction is made.
- **Pre-registration and freezing.** The protocol, its acceptance criteria and the scripts were fixed and hashed with SHA-256 before the test data were seen, and later changes were made only as numbered amendments with their own criteria.

1.7 Explainability and Attacker Modelling

Explainable artificial intelligence makes model decisions understandable. SHAP expresses a single prediction as the sum of a base value φ_0 and one contribution φ_i per feature, computed from Shapley values of cooperative game theory:

$$f(\mathbf{x}) = \phi_0 + \sum_{i=1}^{|F|} \phi_i \tag{1.15}$$

Per-alert explanations of this kind let an analyst check an individual decision; in Graduation Project 2 they will be computed on the vote share of Model E. Attacker behaviour can in turn be modelled as a Markov chain over states such as ATT&CK tactics, in which the next state depends only on the current one:

$$P(X_{t+1} = s_j \mid X_t = s_i, X_{t-1}, \ldots, X_0) = P(X_{t+1} = s_j \mid X_t = s_i) = p_{ij} \tag{1.16}$$

When the defender can act, the interaction becomes a Markov decision process, and Q-learning estimates the value of each action from experience:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \tag{1.17}$$

where α is the learning rate, γ the discount factor and r the reward. The reference study used this formulation inside Cowrie. The present framework uses the Markov chain for intent prediction in Graduation Project 2, but keeps the escalation decision in a supervised classifier and an explicit policy, because both can be validated against labelled data and audited decision by decision.

1.8 Containers and Their Isolation

A container is an isolated group of processes that shares the host's kernel. **Namespaces** give it its own process identifiers, network interfaces, mount points and user identifiers, and **control groups** limit its CPU, memory and process usage; Docker adds images, a managing daemon and an API that a Python orchestrator can drive. Containers start in seconds rather than the tens of seconds a virtual machine needs, many fit on one host, and each can be destroyed after use and recreated from a clean image. The price is the shared kernel: container runtimes have had escape vulnerabilities as recently as November 2025. The decoys therefore combine several isolation mechanisms (Table 1.5).

Table 1.5: Container isolation mechanisms and their use in the decoys

Mechanism	What it limits	Use in the decoys
User namespace (userns-remap)	Container root is an unprivileged host user	Enabled for the Docker daemon
Capabilities, no-new-privileges	Root privileges split into separate rights	All dropped; a few added back per service
Read-only root file system	Persistent changes to the image	Enabled; tmpfs with noexec where needed
cgroup limits	Memory, CPU and number of processes	Fixed limits per decoy
Isolated bridge network	Network access	No NAT; outbound and DNS blocked

A process inside a container sees the host's kernel version, so a decoy that offers a shell must be consistent with that kernel: the SSH decoy uses the host's own Ubuntu release, while the web and database decoys, which expose no shell, can use the smaller Alpine Linux. Because the decoy continues a session that began in Cowrie on the same address, both must look like one host: the decoy carries Cowrie's SSH host keys and version string, Cowrie's emulated file system is built from the decoy, and the attacker's account is created in the decoy before the redirect. Because a decoy still needs a moment to start its services, one ready decoy per service, a **warm standby**, is kept to remove this delay from the attacker's path.

1.9 Moving Target Defense

Moving Target Defense (MTD) changes selected properties of a system over time so that the information an attacker gathers during reconnaissance goes stale. Any MTD strategy must decide what moves, when and how, and every change has a cost. MTD and deception complement each other and are increasingly orchestrated together, either by a learned policy or by declarative rotation of traps. The framework applies MTD to the deception layer itself: at randomised intervals, and only when a decoy has no open connection, it changes the content an attacker can collect, namely files and lure credentials, accounts and the set of

watched ports that answer. The identity of the host (host name, release, kernel and SSH host keys) stays fixed, because a real server does not change it between two visits, and a decoy profile stays pinned to an attacker for as long as the attacker is engaged.

1.10 Small Language Models in Deception

Language models based on the Transformer architecture can generate varied files, logs and records. Small instruction-tuned models such as Qwen2.5 (1.5 and 3 billion parameters) and Phi-3.5-mini (3.8 billion) run on an ordinary processor when their weights are quantised to four bits and served by the llama.cpp engine. LLM-based honeypots that answer commands directly can be convincing, but small models need seconds per command and sometimes invent inconsistent output, and a model that reads attacker input is open to prompt injection. The framework therefore generates decoy content offline, from templates filled only with profile fields, checks it against the decoy profile and falls back to static content when a check fails.

Language models have also changed the attacker side: autonomous agents now carry out multi-step intrusions, and they read and trust the text they encounter, which makes them vulnerable to planted content. Such agents fit neither the Bot nor the Human class well, so the framework treats them as a known limitation of timing-based classification to be measured in the red-team evaluation.

Chapter 2: Design of the Proposed Solution

2.1 Introduction

This chapter presents the design of the proposed solution: the architecture and its design decisions (Section 2.2), the way the design realises the objective (Section 2.3), the flow charts of the operation sequence (Section 2.4), the design parameters (Section 2.5) and the pseudo-code of the programs (Section 2.6). The programs of Graduation Project 2 will be designed in that phase, and measured results will be reported in a later chapter.

2.2 Architecture of the Proposed Solution

Figure 2.1 shows the architecture at the end of Graduation Project 1; the planned layer of Graduation Project 2 is dashed. Every connection from the attacker first meets the Netfilter rules on the external interface. By default, SSH traffic on port 22 is redirected to Cowrie on port 2222. When the orchestrator escalates an attacker, or an attacker probes a watched port, the orchestrator adds a DNAT rule for that source address so that the attacker's new connections reach a decoy container.

The architecture has four layers. **Layer 1, sensing and context,** comprises Cowrie, the frozen classifier, Zeek and the attack surface analysis. **Layer 2, decision,** is the orchestrator, an event loop that keeps a context per session and applies the policy Λ. **Layer 3, action,** comprises the decoy manager, the redirector, the MTD rotator and the offline content generator. **Layer 4, explanation and forensics,** belongs to Graduation Project 2 and reads the SQLite database in which every decision is recorded.

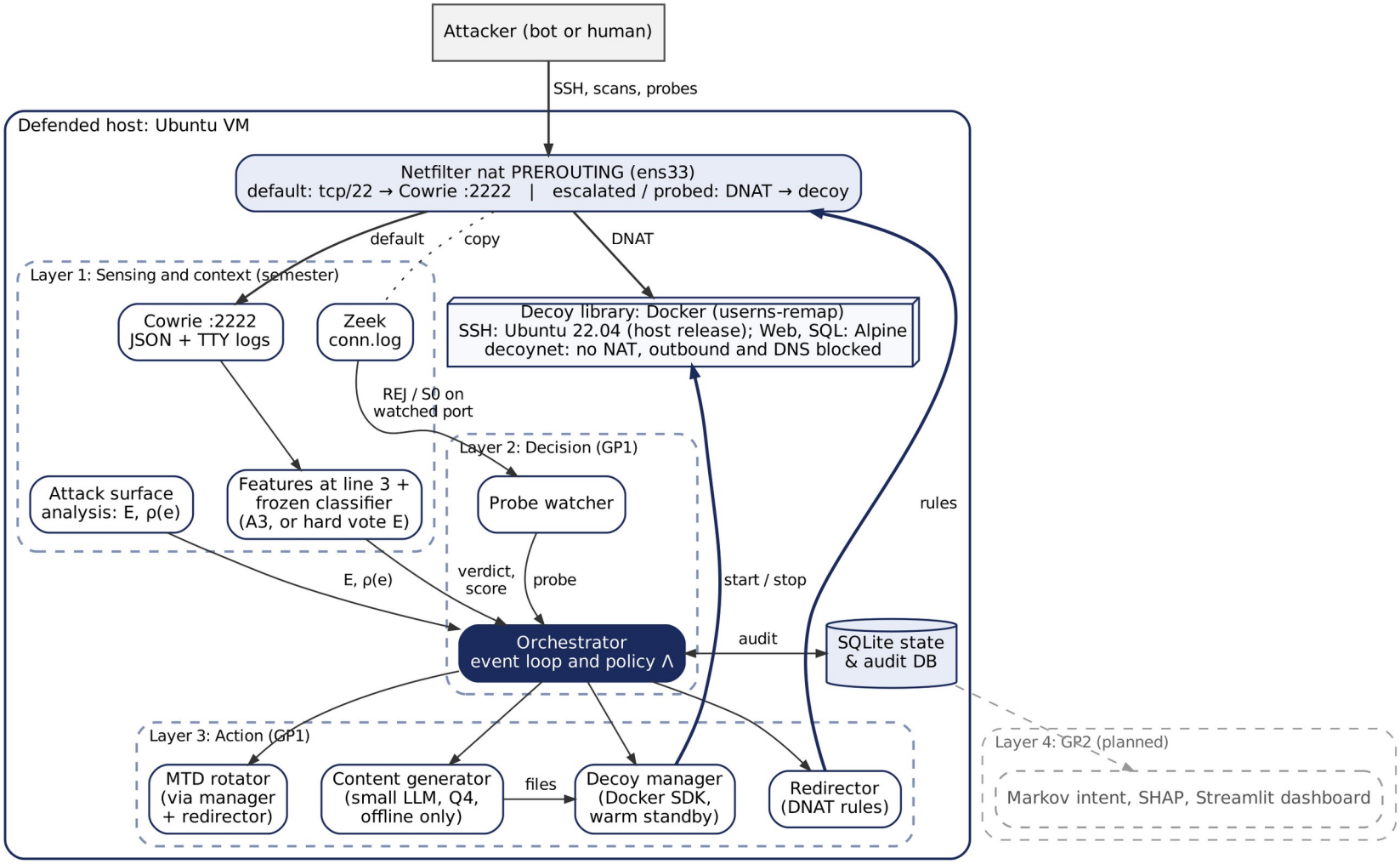


Figure 2.1: Architecture of the proposed solution at the end of GP1 (the GP2 layer is dashed)

2.2.1 Design Decisions

- **Escalation applies to the attacker's next connections.** An established connection cannot be moved without breaking it, so the DNAT rule matches only new connections and the current session stays in Cowrie.
- **The classifier is frozen and decides once.** The model, k and θ come from the configuration, the model's SHA-256 hash is checked on loading, and each session is classified once, at line k, with the same feature code as in training.
- **Self-morphing is limited.** Only watched ports that the host does not serve can trigger a decoy, probe events are de-duplicated, and each address may trigger only a capped number of decoys.
- **Decoys are isolated in depth.** Every decoy runs with user-namespace remapping, dropped capabilities, no-new-privileges, a read-only root file system and resource limits, on an isolated network without NAT, outbound traffic or DNS. A preflight check refuses any deployment that lacks one of these protections.
- **Decoys are consistent with their host and with Cowrie.** The SSH decoy runs the host's own Ubuntu release with Cowrie's host keys, version string and host name, and records every input line through a session wrapper; Cowrie's emulated file system is built from the decoy. The web and database decoys run Alpine Linux and carry the same host name.
- **The attacker's account carries over.** Before the DNAT rule is added, the SSH decoy receives the user name and password the attacker used in Cowrie, so the next login works as it did before.
- **Start-up latency is hidden** by one warm standby decoy per service, and **generated content is produced offline**, so neither container start-up nor the language model appears in an attacker's session.
- **Evidence comes before removal, and every decision is recorded with its reason** in the SQLite database; every Netfilter rule carries an identifying comment.

2.3 How the Solution Realises the Objective

Figure 2.2 decomposes the project objective into six sub-objectives and shows, for each one, the component that realises it and the check that will show whether it was achieved. O1 and O2 provide the two kinds of context, the attacker and the host; O3 turns them into real-time decisions; O4 to O6 concern the quality of the response, which has to be convincing, unpredictable, and safe and accountable.

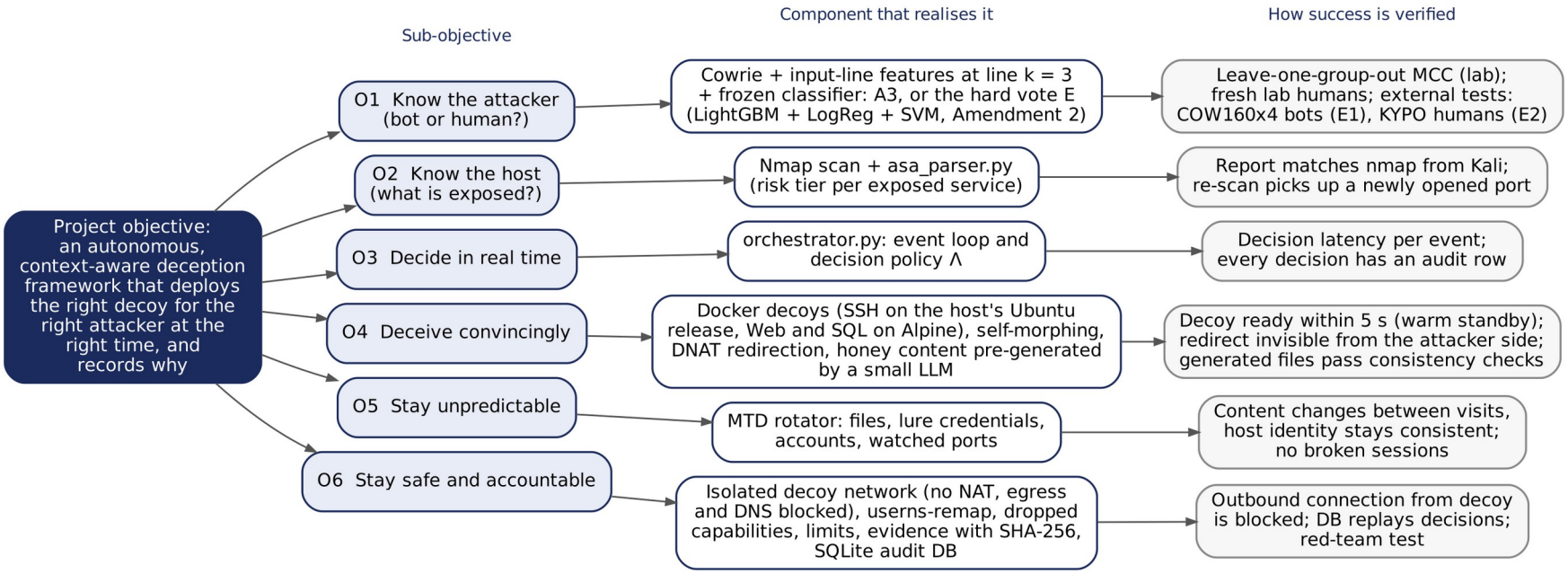


Figure 2.2: Realisation of the project objective: sub-objectives, components and verification

2.4 Flow Charts of the Operation Sequence

Four flow charts describe the order of operations, using the symbols of Table 2.1.

Table 2.1: Flow-chart symbols

Symbol	Meaning
Ellipse	Start or end of a flow
Rectangle	Processing step
Double-bordered rectangle	Predefined process detailed in another figure
Diamond	Decision with labelled outgoing branches
Parallelogram	Input or waiting for input
Cylinder	Write to the SQLite database
Folded page	Output file
Circle with a letter	On-page connector
Dashed rounded frame	Group of steps that belong together

2.4.1 Runtime Loop of the Orchestrator

At start-up the orchestrator loads its configuration, the frozen classifier and the decoy catalogue, scans the host and installs the baseline rules, then waits for events (Figure 2.3). When a session reaches input line 3 it is classified; a Bot stays in Cowrie, and a Human receives a decoy chosen by Λ and deployed as in Figure 2.5. Probes, the MTD timer and the re-scan timer are handled in the same loop, and every branch ends by recording the decision.

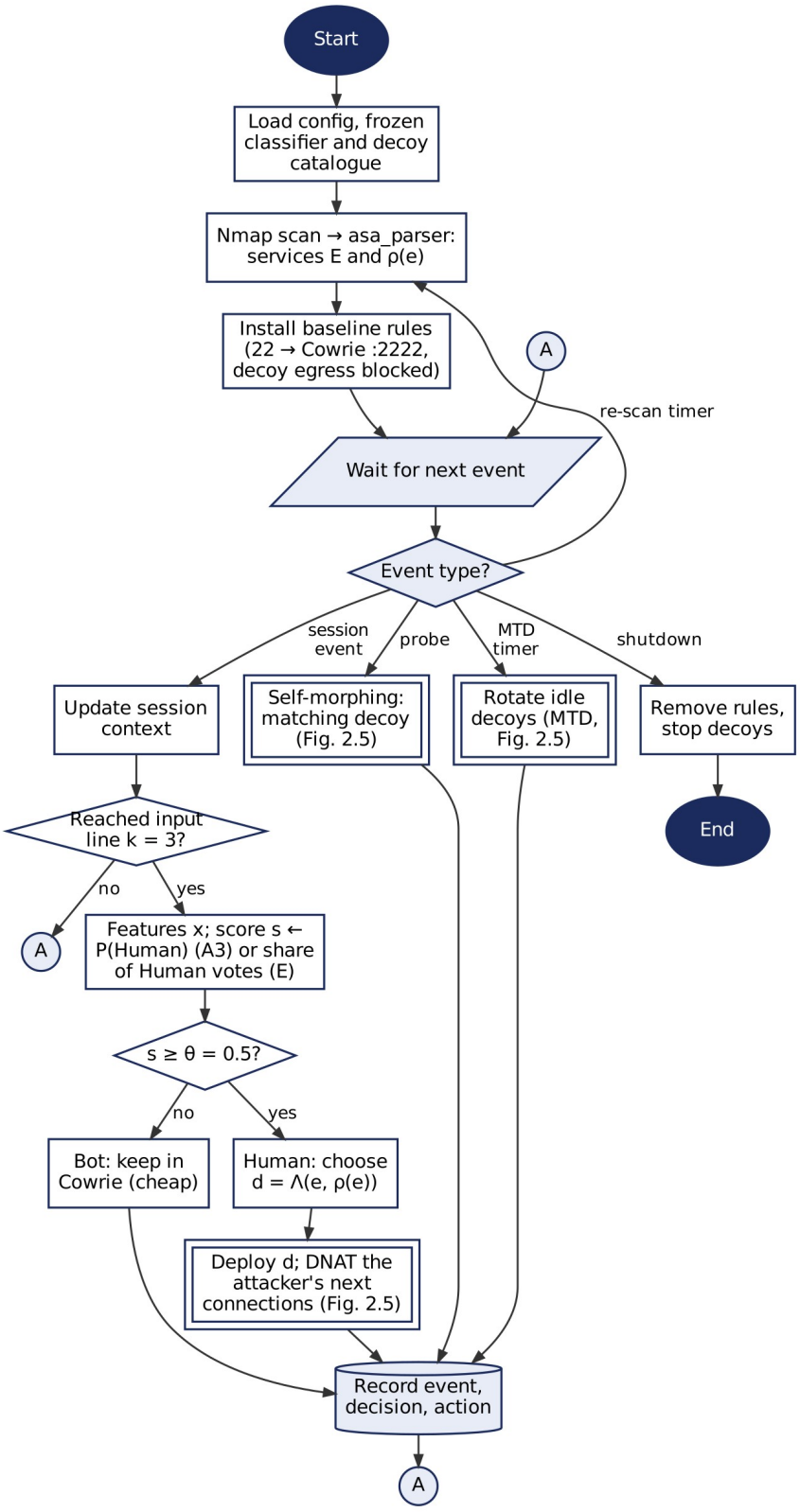


Figure 2.3: Flow chart of the runtime operation sequence

2.4.2 Data, Training and Validation Pipeline

Figure 2.4 shows the offline pipeline of the classifier. The protocol is fixed and hashed first. The left branch builds the laboratory data set (artefacts removed, labels from login names),

selects the configuration by LOGO and freezes the models. The right branch draws unused days of COW160x4, verifies and inspects them without predictions, reviews recurring sequences blind, and only then scores the frozen models; the KYPO humans are scored the same way.

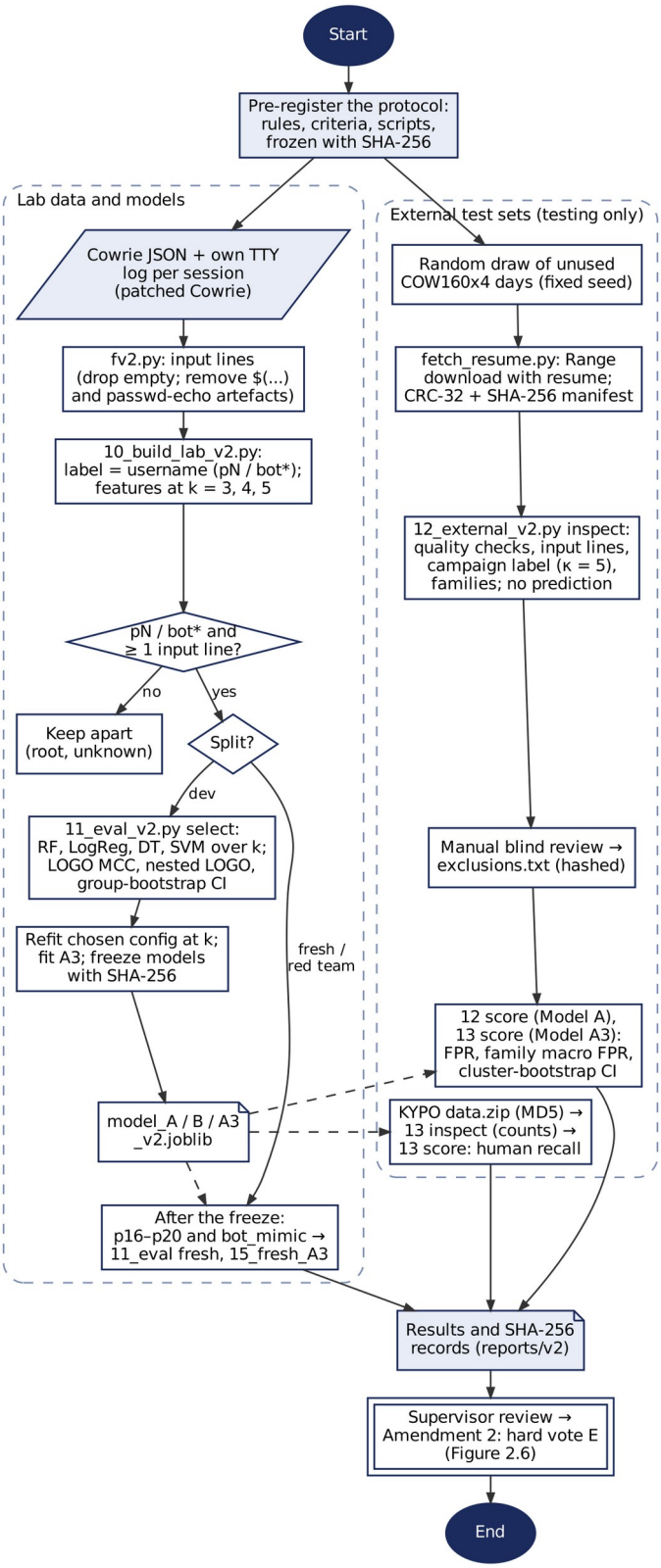


Figure 2.4: Flow chart of the data, training and validation pipeline (pre-registered protocol v2)

2.4.3 Life Cycle of a Decoy

A request for an address and a service reuses an existing decoy or the warm standby; otherwise capacity is checked, a profile and pre-generated content are prepared, and a

hardened container is started on the isolated network (Figure 2.5a). The DNAT rule is added only if the decoy answers within five seconds and, for SSH, after the attacker's account has been created in it. A periodic monitor retires idle or expired decoys, collecting evidence first, and recreates decoys whose MTD period has expired when they have no open connection (Figure 2.5b).

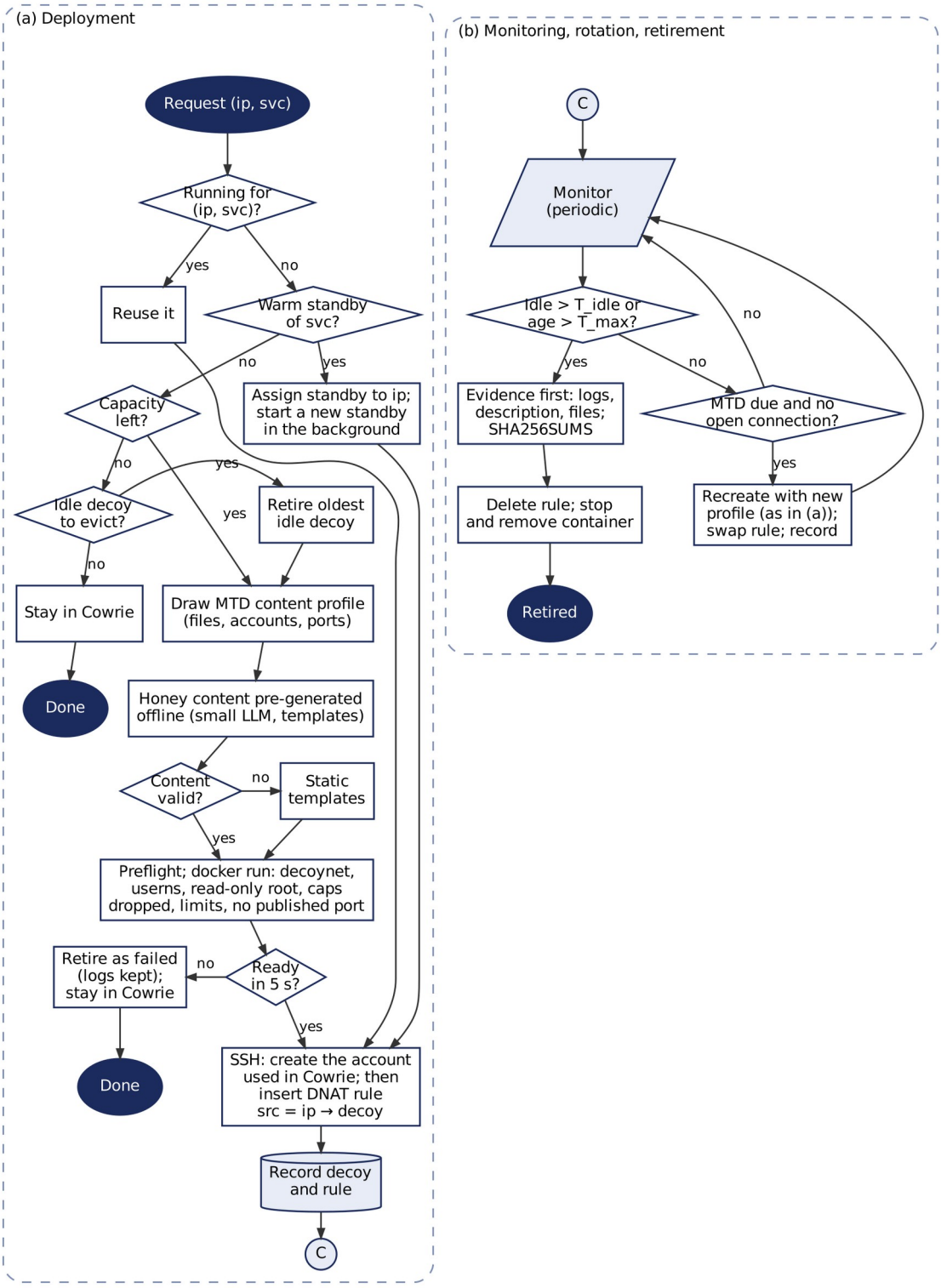


Figure 2.5: Flow chart of the decoy life cycle: (a) deployment, (b) monitoring, rotation and retirement

2.4.4 The Voting Ensemble (Model E)

In Model E (Figure 2.6a) the three timing features of the first three lines go to LightGBM, logistic regression and an RBF SVM, and the session is classified as Human when at least two members agree. The adoption procedure of Amendment 2 (Figure 2.6b) replaces the

frozen single model only if both pre-registered criteria hold: a false-human rate of at most 10% on bots (E1) and a human recall of at least 0.80 (E2).

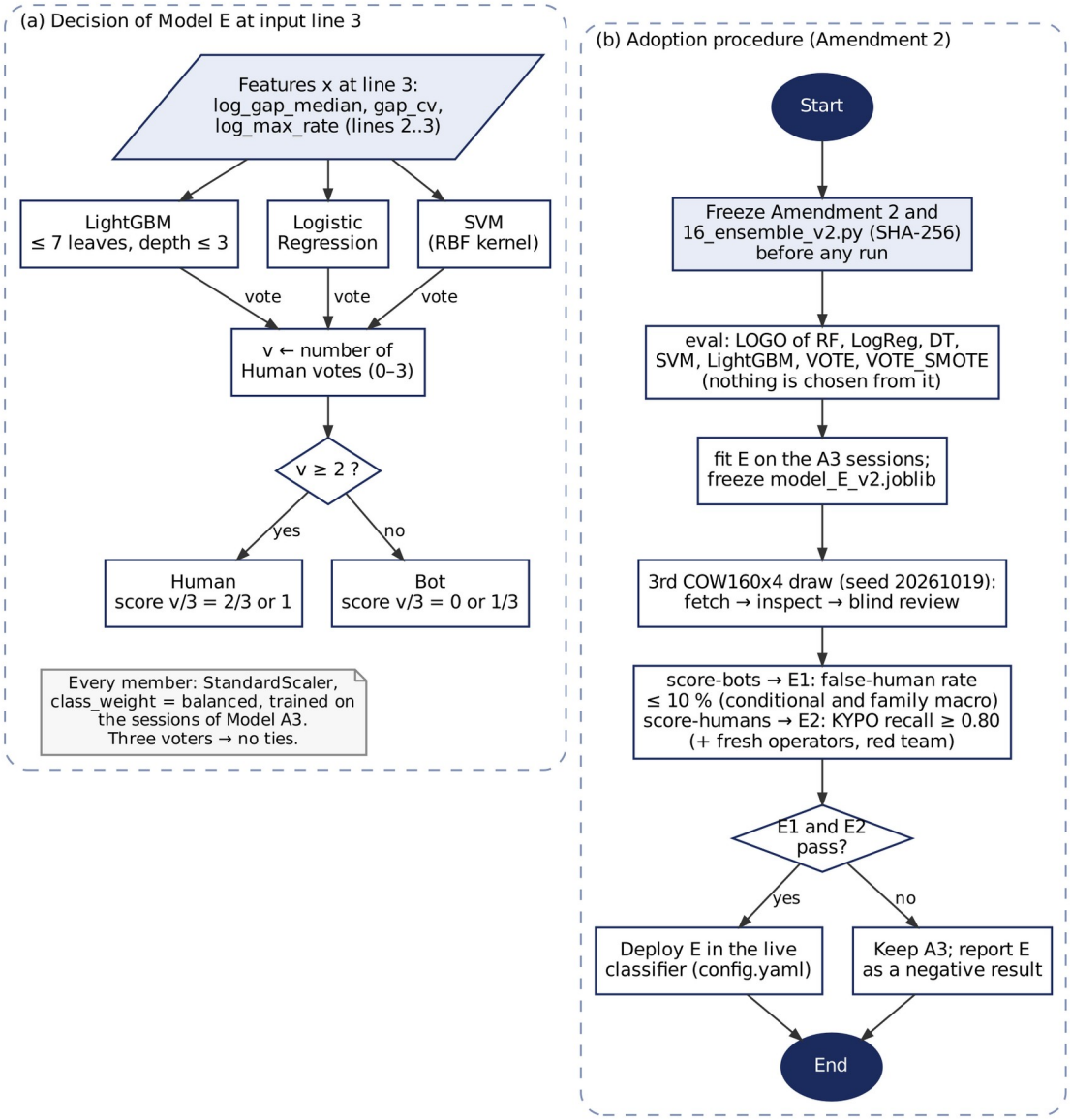


Figure 2.6: Model E, the hard-vote ensemble of Amendment 2: (a) decision at input line 3, (b) adoption procedure

2.5 Design Parameters

Table 2.2 lists the design parameters. The classifier parameters were fixed by the pre-registered protocol; the runtime values are initial values that may be adjusted during integration.

Table 2.2: Design parameters and initial values

Symbol / name	Meaning	Initial value
k	Input line at which a session is classified	3 (chosen among 3, 4 and 5 by LOGO MCC before testing)

Symbol / name	Meaning	Initial value
θ	Decision threshold on the score s	0.5; for Model E, at least 2 of 3 votes
θ_{high}	Threshold for the richest decoy profile	0.8; for Model E, $s = 1$
Members of E	Models that vote in Model E	LightGBM, logistic regression, SVM (RBF); standardisation and balanced class weights
LightGBM	Small-data configuration	200 trees, learning rate 0.05, at most 7 leaves, depth at most 3, at least 10 sessions per leaf, seed 42
E1, E2	Acceptance criteria of Model E	E1: false-human rate ≤ 10% (conditional and family macro); E2: KYPO recall ≥ 0.80
κ	Distinct source addresses that define a campaign bot	5
B	Bootstrap resamples for 95% intervals	2,000, seed 42
W, T_w	Watched ports and probe de-duplication window	{3306, 5432, 6379, 8080}; 60 s
MAX_DECOYS	Decoys running at the same time	3, plus 1 warm standby per service
Decoy resources	Limits of each container	256 MB memory without swap, 0.5 CPU, process limit
decoynet	Isolated decoy network	172.30.0.0/24, no masquerade, DNS 192.0.2.1 (reserved address that never answers)
T_{idle}, T_{max}	Idle time and maximum lifetime of a decoy	10 min, 60 min
P_{MTD}	Mean period between rotations	30 min ± 10 min

2.6 Pseudo-Code of the Software

This section gives the pseudo-code of every program of the semester phase and of Graduation Project 1 (Table 2.3). Section 2.6.1 covers the classifier programs (Algorithms 2.1 to 2.10) and Section 2.6.2 the runtime programs (Algorithms 2.11 to 2.19). In the notation, ← denotes assignment, ▷ a comment, indentation a block and ∅ an empty result; every algorithm states its file, phase, inputs and outputs.

Table 2.3: Software modules and their pseudo-code

Algorithm	Name	Program	Status
2.1	ATTACK-SURFACE-ANALYSIS	`scripts/asa_parser.py`	Implemented
2.2	SESSION-FEATURE-EXTRACTION	`scripts/fv2.py`	Implemented
2.3	PROVENANCE-LABELLING	`scripts/10_build_lab_v2.py`	Implemented
2.4	LEAVE-ONE-GROUP-OUT-EVALUATION	`scripts/11_eval_v2.py`	Implemented
2.5	FINAL-MODEL-TRAINING	`11_eval_v2.py`, `13_external_human.py`, `16_ensemble_v2.py`	Implemented
2.6	BOT-SESSION-GENERATOR	`kali/bot_v2.py`	Implemented

Algorithm	Name	Program	Status
2.7	FETCH-EXTERNAL-SUBSET	`scripts/fetch_resume.py`	Implemented
2.8	EXTERNAL-VERIFICATION-AND-VALIDATION	`12_external_v2.py`, `13_external_human.py`	Implemented
2.9	RED-TEAM-PTY-BOT	`kali/bot_v2.py` (class `bot_mimic`)	Implemented
2.10	HARD-VOTE-ENSEMBLE	`scripts/16_ensemble_v2.py`	Implemented
2.11	ORCHESTRATOR-MAIN-LOOP	`orchestrator/ orchestrator.py`	Loop implemented
2.12	CLASSIFY-LIVE	`orchestrator/ live_classify.py`	Implemented
2.13	DECIDE (policy Λ)	`orchestrator.py`, `policy.py`	Planned
2.14	DECOY-MANAGER	`orchestrator/ decoy_manager.py`	Implemented
2.15	REDIRECTOR	`redirector.py`	Planned
2.16	PROBE-WATCHER	`probe_watcher.py`	Planned
2.17	MTD-ROTATE	`mtd.py`	Planned
2.18	GENERATE-CONTENT	`content_gen.py`	Planned
2.19	STATE-DB	`orchestrator/state_db.py`, `schema.sql`	Implemented

2.6.1 Classifier Programs

Algorithm 2.1 assigns each open service in an Nmap report a risk tier, giving the host context E with $\rho(e)$.

Algorithm 2.1: ATTACK-SURFACE-ANALYSIS
File: scripts/asa_parser.py Phase: Semester (implemented)
Input : Nmap XML report R (nmap -sS -sV -oX R <host>)
Output: set E of exposed services, each with risk tier ρ(e); CSV report

```
HIGH ← {ssh, telnet, ftp, rsh, rlogin}
MEDIUM ← {http, https, smtp, pop3, imap, rdp, vnc}
E ← ∅
for each host h in R do
    for each port p of h do
        if state(p) ≠ open then continue
        e ← (port(p)/protocol(p), service name s, product + version)
        if port(p) = 2222 then ρ(e) ← HONEYPOT                    ▷ Cowrie's own port
        else if s ∈ HIGH then ρ(e) ← HIGH
        else if s ∈ MEDIUM then ρ(e) ← MEDIUM
        else ρ(e) ← LOW
        E ← E ∪ {e}
write E to CSV and print it sorted by ρ
if some e ∈ E has ρ(e) = HIGH then
    recommend deception for e   (e.g. REDIRECT tcp/22 → 2222)
return E
```

Algorithm 2.2, shared by the laboratory, external and live paths, reconstructs input lines without artefacts and computes the features of lines 1 to k.

Algorithm 2.2: SESSION-FEATURE-EXTRACTION

```
File  : scripts/fv2.py (shared by lab, external and live paths)
Phase : Classifier rebuild, Oct 2026 (implemented)
Input : command.input events [(t, x)] of one session, login time t0 (may be missing),
        TTY records of the session (may be missing), decision line k
Output: input lines L; JSON features; TTY features (NaN when missing); or NOT_REACHED
```

```
▷ input lines: remove Cowrie artefacts
E ← events sorted by t, text stripped; drop empty texts
L ← [ ]
for each (t, x) in E do
    if last line of L contains $( or ` and norm(x) ⊂ that line and t − t_prev ≤ 1 s
        then skip x                                                    ▷ $(...) substitution artefact
    else append (t, x) to L
drop (t, x) from L if x starts with "Changing password for " and the previous
    line pipes passwd into sh/bash within 1 s                                  ▷ passwd-echo artefact
if |L| < k then return NOT_REACHED                                  ▷ no decision for this session
▷ JSON features on lines 1..k only
g ← (t2 − t1, …, tk − tk−1)
log_first_latency ← log10(t1 − t0)                                          ▷ NaN when t0 is missing
log_gap_median ← log10(median(g)) ; gap_cv ← std(g) / mean(g)
log_max_rate ← log10(max_i |x_i| / gap_before_i)                  ▷ chars/s; line 1 only if t0 is known
▷ TTY features on keystrokes up to tk + 0.1 s (Model B)
if there is no keyboard-input write then TTY features ← NaN                          ▷ missing, never 0
else single_char_ratio ← share of input writes that carry one byte
    log_key_gap_median ← log10(median gap between writes inside a line)
    edit_keys_per_line ← (BS, DEL, TAB, Ctrl-U, Ctrl-W, ESC) / k
return (L, features)                                                ▷ gaps and logs floored at 1 ms
```

Algorithm 2.3 labels laboratory sessions by provenance only and assigns each human operator or bot class to one validation group. Human sessions were typed manually over SSH by trained operators, each under a separate account.

Algorithm 2.3: PROVENANCE-LABELLING

```
File: scripts/10_build_lab_v2.py    Phase: Classifier rebuild, Oct 2026 (implemented)
Input : Cowrie JSON logs and TTY directory of the lab; fresh users; red-team users
Output: lab_v2_sessions.csv and lab_v2_features.csv (label, group, split, features at k)
```

```
for each session s with at least one command event do
    if user(s) matches ^p[0-9]+$ then label ← Human
    else if user(s) matches ^bot[\w-]*$ then label ← Bot
    else continue                                                  ▷ root (operator tests) and unknown users
    group(s) ← user(s)                                  ▷ one LOGO group per human operator or bot class
    split(s) ← fresh if user(s) ∈ {p16..p20}; redteam if bot_mimic; else dev
    tty_status(s) ← own TTY file? (ok / dedup_deleted / no_log_closed / missing)
    for each k ∈ {3, 4, 5}: if s reaches line k then add the features of Algorithm 2.2
save lab_v2_sessions.csv and lab_v2_features.csv
▷ the label comes from provenance only; no feature value is used
```

Algorithm 2.4 selects the decision line and the algorithm by LOGO MCC and estimates the effect of the selection with nested LOGO.

Algorithm 2.4: LEAVE-ONE-GROUP-OUT-EVALUATION

File: scripts/11_eval_v2.py (mode select) Phase: Classifier rebuild, Oct 2026 (implemented)
Input : dev rows of lab_v2_features.csv; Model A = JSON features,
Model B = JSON + TTY features; K = {3, 4, 5}
Output: selection.json, selection_table.csv (MCC and 95 % CI), nested-LOGO estimate

```
for each model M ∈ {A, B} do
    P ← dev sessions that reach line 5 (B: with keyboard input)              ▷ same sessions for every k
    for each k ∈ K and each algorithm m ∈ {RF, LogReg, DT, SVM} do
        ŷ ← LOGO-PREDICT(P at k, m) ; store MCC(y, ŷ) and its group-bootstrap CI
    report the majority baseline and the simple rate rule the same way
    (k*, m*) ← highest LOGO MCC                                  ▷ ties: smaller k, then RF, LogReg, DT, SVM
    for each group g do                                                                        ▷ nested LOGO
        (k_g, m_g) ← the same choice made without g ; predict g with (k_g, m_g)
    report nested MCC with its 95 % group-bootstrap CI (B = 2000, seed 42)
▷ Amendment 2 adds LightGBM and the hard vote E: Algorithm 2.10

procedure LOGO-PREDICT(D, m)
    for each group g in D do                                          ▷ one human operator or one bot class
        fit StandardScaler → m (class_weight = balanced, seed 42) on D \ g
        ŷ[g] ← m(g)
    return ŷ
```

Algorithm 2.5 refits and freezes the selected models, including Model A3 and Model E, and records their SHA-256 hashes.

Algorithm 2.5: FINAL-MODEL-TRAINING

File : scripts/11_eval_v2.py, 13_external_human.py, 16_ensemble_v2.py
Phase : Classifier rebuild, Oct 2026 (implemented)
Input : (k*, m*) of Algorithm 2.4 for Models A and B; lab_v2_features.csv
Output: models/model_A_v2, model_B_v2, model_A3_v2 and model_E_v2 (.joblib); SHA-256 records

```
for each model M ∈ {A, B} do
    D ← dev sessions at line k* with no missing feature (B: with keyboard input)
    fit StandardScaler → m* on D
    save model_M_v2.joblib with k*, m*, features, n_train and the table hash
▷ Model A3: for data without a login time (e.g. KYPO)
D3 ← the sessions of Model A ; features without log_first_latency,
    rate taken over lines 2..k
fit the algorithm of Model A on D3 ; save model_A3_v2.joblib
▷ Model E (Amendment 2): hard vote of LightGBM, LogReg and SVM
fit E on D3 ; save model_E_v2.joblib                                  ▷ Algorithm 2.10, stage fit
write the SHA-256 of every model and of the feature table
▷ frozen: no feature, threshold, k or model changes after test data are seen
```

Algorithm 2.6 generates bot sessions with known provenance and several pacing styles from the Kali machine.

Algorithm 2.6: BOT-SESSION-GENERATOR

```
File: kali/bot_v2.py (runs on Kali)    Phase: Classifier rebuild, Oct 2026 (implemented)
Input : target host, class c ∈ {bot_exec, bot_rtt, bot_sleep, bot_slow, bot_burst,
        bot_mimic}, number of sessions n, seed
Output: n Cowrie sessions of class c with known provenance (username = c)
```

```
rng ← Random(c, seed)
repeat n times
    S ← 1–15 commands drawn without replacement from a pool of 35                    ▷ bot_burst: 3–15
    if c = bot_exec then run 1–6 commands joined by ';' as one exec request ; next
    open SSH as user c with paramiko ; chan ← invoke_shell() ; append exit to S
    for each line x in S do
        if c = bot_burst then send 2–5 lines in one write, then wait for the prompt
        else if c = bot_mimic then think (median 4 s), type x key by key              ▷ Algorithm 2.9
        else wait by class, then send x in one write:
            bot_rtt: per-session delay 0.05–0.8 s (±10 %)
            bot_sleep: fixed wait D ∈ {2, 5, 10, 20} s (±1 %)
            bot_slow: cost of previous command (0.3 / 1 / 3 s) × slowness 1–10 (±20 %)
        read the output until the prompt returns
    close the connection
▷ 20 sessions per class with seed 1; bot_mimic is used for red-team tests only
```

Algorithm 2.7 downloads only the selected days of the external COW160x4 archive and verifies every file.

Algorithm 2.7: FETCH-EXTERNAL-SUBSET

```
File: scripts/fetch_resume.py    Phase: Classifier rebuild, Oct 2026 (implemented)
Input : URL of data_all.zip (Zenodo 21260400), list of days, output directory
Output: verified day files *.json.gz + manifest.json (CRC-32, SHA-256, source, DOI)
```

```
Z ← central directory of the remote zip, read with HTTP Range requests
for each requested day d do
    i ← entry of d in Z ; if missing then stop with an error
    if the output file exists and its CRC-32 = i.CRC then continue                  ▷ already complete
    read the local header ; a ← offset + 30 + name length + extra length
    GET-RANGE(a, a + compressed size − 1, d.part)
    inflate d.part ; if CRC-32 ≠ i.CRC then delete d.part and stop
    write the day file ; delete d.part
write manifest.json: day, member, CRC-32, SHA-256, URL, DOI, retrieval time

procedure GET-RANGE(a, b, out)
    repeat up to 200 attempts
        request bytes [a + size(out), b] and append them to out
        on a network error wait 10 s                                  ▷ resume from the last byte
```

Algorithm 2.8 inspects an external sample without predictions, supports the blind review, and then scores the frozen models.

Algorithm 2.8: EXTERNAL-VERIFICATION-AND-VALIDATION

```
File  : scripts/12_external_v2.py (bots), scripts/13_external_human.py (KYPO)
Phase : Classifier rebuild, Oct 2026 (implemented)
Input : confirmatory day files (drawn with a fixed seed), frozen models, κ = 5;
        KYPO data.zip (MD5 checked)
Output: inspect.json, sequences_for_inspection.txt; score.json, per_family.csv
```

```
▷ stage inspect: no prediction is made
count lines, malformed lines, missing fields and duplicate events
keep complete sessions (connect and closed) with ≥ 1 input line (Algorithm 2.2)
seq(s) ← template hash of the line sequence ; family(s) ← template of line 1
campaign(s) ← |source addresses sharing seq(s)| ≥ κ
write every campaign sequence with ≥ 5 sessions at line k for manual reading
▷ blind review: a human-operated sequence goes to exclusions.txt (hashed)
▷ stage score
remove the excluded sequences ; R ← campaign sessions that reach line k
predict R with the frozen model                                  ▷ A in 12, A3 in 13, E next to A3 in 16
conditional FPR ← predicted Human / |R|
operational FPR ← predicted Human / campaign sessions with ≥ 1 line
macro FPR ← mean per-family FPR over families with ≥ 5 sessions at line k
95 % CIs by cluster bootstrap over families (B = 2000, seed 42)
▷ KYPO humans (13): bash commands of each trainee, sorted, duplicates removed
exclude the training with whole-second timestamps from the primary test
recall ← trainees predicted Human / trainees that reach line k                          ▷ bootstrap CI
```

Algorithm 2.9 is the red-team bot that types like a person; its sessions are used only to test the classifier.

Algorithm 2.9: RED-TEAM-PTY-BOT

```
File  : kali/bot_v2.py, class bot_mimic (runs on Kali)
Phase : Classifier rebuild, Oct 2026 (implemented)
Input : target, user bot_mimic, human-paced timing model
Output: adversarial sessions that imitate human typing (red-team test only)
```

```
open SSH with a PTY (term = xterm) and invoke_shell()
S ← 1–15 commands drawn from the pool of Algorithm 2.6, then exit
for each command c in S do
    sleep(log-normal think time, median ≈ 4 s)
    for each character ch of c do
        with probability 0.05 send a wrong key, then DEL (backspace)
        send ch ; sleep(log-normal key gap, median ≈ 0.18 s)
    send carriage return ; read the output until the prompt returns
close the session
▷ the share of these sessions classified as Human is reported in the results
▷ as a limitation of timing-based detection, whatever its value
```

Algorithm 2.10 implements Amendment 2: it defines, compares, freezes and externally tests Model E.

Algorithm 2.10: HARD-VOTE-ENSEMBLE (Model E)

```
File: scripts/16_ensemble_v2.py    Phase: Amendment 2, 19 Oct 2026 (implemented)
Input : dev sessions of Model A3 (k = 3), frozen A3, raw Cowrie log; KYPO data;
        fresh and red-team sessions; third COW160x4 draw (seed 20261019)
Output: eval.json (LOGO table), model_E_v2.joblib + SHA-256, humans.json (E2),
        bots_confirmatory3.json (E1)
```

```
▷ Model E: every configuration below is fixed in Amendment 2
members ← LightGBM (200 trees, ≤ 7 leaves, depth ≤ 3, ≥ 10 sessions per leaf),
    LogReg, SVM (RBF) ; each = StandardScaler → model, class_weight = balanced
E(x) ← Human if at least 2 of the 3 members say Human                    ▷ hard vote, no ties
▷ stage eval: plain LOGO, nothing is chosen from it
D ← A3 sessions with the A3 features rebuilt from the raw Cowrie log
if refitting A3's frozen configuration on D ≠ frozen A3 then stop                  ▷ same data
for each c ∈ {RF, LogReg, DT, SVM, LightGBM, VOTE, VOTE_SMOTE, majority, rule} do
    ŷ_c ← LOGO-PREDICT(D, c) ; report MCC, group-bootstrap CI, per group            ▷ Alg. 2.4
VOTE_SMOTE: members without class weights, SMOTE inside each training fold
report member diversity: disagreement, double fault, Q statistic
▷ stage fit
fit E on D ; save model_E_v2.joblib ; record its SHA-256                  ▷ never overwritten
▷ stages score-humans and score-bots — E is scored next to A3
E2 ← (recall of E on the 249 primary KYPO trainees ≥ 0.80)
report fresh p16–p20 TPR and bot_mimic TNR (operator bootstrap)                ▷ descriptive
R ← third COW160x4 draw after fetch, inspect and blind review (Alg. 2.7, 2.8)
E1 ← conditional and family-macro false-human rates of E on R ≤ 10 %
if E1 and E2 then use E in CLASSIFY-LIVE (Alg. 2.12) else keep A3
```

2.6.2 Graduation Project 1 Runtime Programs

Algorithm 2.11 is the main loop of the orchestrator, which dispatches every event and records each decision.

Algorithm 2.11: ORCHESTRATOR-MAIN-LOOP

```
File: orchestrator/orchestrator.py    Phase: GP1 (loop implemented; actions planned)
Input : configuration (θ, k, T_idle, T_max, P_MTD, MAX_DECOYS),
        frozen classifier, decoy catalogue
Output: deception actions and one audit record per decision
```

```
db ← STATE-DB.open() ; model ← load(frozen classifier) ; cat ← load(decoys.yaml)
(E, ρ) ← ATTACK-SURFACE-ANALYSIS(nmap scan of the host)                    ▷ Algorithm 2.1
REDIRECTOR.install_baseline()                                              ▷ Algorithm 2.15
DECOY-MANAGER.preflight() ; DECOY-MANAGER.fill_standby()                   ▷ Algorithm 2.14
Q ← event queue fed by: tail of cowrie.json, PROBE-WATCHER, timers
loop
    ev ← Q.get()
    switch type(ev)
        case COWRIE_EVENT:
            ctx ← context[ev.session] ; ctx.update(ev)
            if ctx.verdict = ∅ and ctx has reached input line k then
                (verdict, p) ← CLASSIFY-LIVE(ctx, model)                   ▷ Algorithm 2.12
                action ← DECIDE(ctx, verdict, p, E, ρ)                     ▷ Algorithm 2.13
                EXECUTE(action) ; db.log_decision(ev, ctx, verdict, p, action)
            if ev is cowrie.session.closed then db.close_session(ctx)
        case PROBE:
            action ← DECIDE-PROBE(ev, E, cat) ; EXECUTE(action) ; db.log_decision(...)
        case MONITOR_TIMER: DECOY-MONITOR()                                ▷ Algorithm 2.14
        case MTD_TIMER:     MTD-ROTATE()                                   ▷ Algorithm 2.17
        case RESCAN_TIMER:  (E, ρ) ← ATTACK-SURFACE-ANALYSIS(...) ; db.log_scan(E)
        case SHUTDOWN:      REDIRECTOR.flush() ; DECOY-MANAGER.remove_all() ; exit loop
procedure EXECUTE(action)
    if action = KEEP_IN_COWRIE or IGNORE then return
    d ← DEPLOY-DECOY(action.ip, action.svc, action.profile, logins(action.ip))   ▷ Algorithm 2.14
    if d ≠ ∅ then REDIRECTOR.add(action.ip, action.svc.port, d)
```

Algorithm 2.12 classifies a live session once it reaches line *k* and returns the verdict with its score.

Algorithm 2.12: CLASSIFY-LIVE

```
File: orchestrator/live_classify.py    Phase: GP1 (implemented); vote branch with Amendment 2
Input : session context ctx (command events, login time, TTY records so far),
        frozen classifier(s), threshold θ, decision line k
Output: verdict ∈ {Bot, Human} and score p, or PENDING
```

```
L ← input lines of ctx (Algorithm 2.2) ; if |L| < k then return PENDING
x ← features at line k, computed with fv2                                  ▷ same code as training
if keyboard input is present and Model B is enabled then model ← B
else model ← the JSON model: E (A3 until Amendment 2)
▷ missing keystroke data stays missing; it is never replaced by 0
if model is a hard vote then                                                          ▷ Model E
    v ← number of members (LightGBM, LogReg, SVM) that say Human
    p ← v / 3                                                              ▷ 0, 1/3, 2/3 or 1
else p ← model.predict_proba(x)[Human]
verdict ← Human if p ≥ θ else Bot                          ▷ θ = 0.5: for E, at least 2 of 3 votes
return (verdict, p)
```

Algorithm 2.13 is the decision policy Λ for classified sessions and for probes.

Algorithm 2.13: DECIDE (decision policy Λ)

```
File: orchestrator.py / policy.py    Phase: GP1 (planned)
Input : context (address ip, service svc), verdict, p, exposed services E with ρ
Output: action ∈ {KEEP_IN_COWRIE, IGNORE, DEPLOY(ip, svc, profile)} with its reason
```

```
procedure DECIDE(ctx, verdict, p, E, ρ)
    if verdict = Bot then return KEEP_IN_COWRIE                ▷ scripts gain nothing from realism
    svc ← ssh                                                          ▷ Cowrie sessions are SSH
    if ρ(svc) = HIGH or p ≥ θ_high then profile ← rich profile of svc      ▷ E: p = 1, all 3 votes
    else profile ← default profile of svc
    return DEPLOY(ctx.ip, svc, profile, reason = {verdict, p, ρ(svc)})
    ▷ applies to the attacker's next connections; the open session stays in Cowrie

procedure DECIDE-PROBE(ev, E, cat)
    svc ← cat.service_for_port(ev.port)                          ▷ e.g. 3306 → sql, 8080 → web
    if svc = ∅ then return IGNORE
    if ev.port is served by a real service in E then return IGNORE                ▷ never shadow it
    if decoys already assigned to ev.ip ≥ per-address cap then return IGNORE
    return DEPLOY(ev.ip, svc, default profile of svc, reason = probe)
```

Algorithm 2.14 is the decoy manager: preflight check, deployment with warm standby, monitoring and evidence-first retirement.

Algorithm 2.14: DECOY-MANAGER (preflight, deploy, monitor, retire)

File: orchestrator/decoy_manager.py (Docker SDK) Phase: GP1 (implemented); warm standby planned
Input : address ip, service svc, profile π, accounts L used by ip in Cowrie;
catalogue C: image, port, capabilities, tmpfs mounts and process limit per service
Output: running decoy d (id, container, address, port) or ∅; evidence directory with SHA256SUMS

```
procedure PREFLIGHT()
    require network decoynet: subnet 172.30.0.0/24, masquerade off
    require the isolation service acdf-net to be active and every image of C present
    if a requirement fails then refuse every deployment                    ▷ no decoy without isolation

procedure DEPLOY-DECOY(ip, svc, π, L)
    if a healthy decoy for (ip, svc) exists then return it
    if a warm standby decoy of svc exists then                                                ▷ planned
        d ← standby ; ADD-LOGINS(d, L) ; assign d to ip ; start a new standby ; return d
    if running decoys ≥ MAX_DECOYS then
        if an idle decoy exists then RETIRE(oldest idle decoy) else return ∅
    PREFLIGHT()
    vol ← GENERATE-CONTENT(svc, π) if prepared, else the image's static honey files       ▷ Algorithm 2.18
    c ← docker.run(C[svc].image, network = decoynet, dns = 192.0.2.1,
                read-only root, tmpfs mounts with noexec, cap_drop = ALL, cap_add = C[svc].caps,
                no-new-privileges, 256 MB without swap, 0.5 CPU, pids = C[svc].pids,
                hostname = host identity, volume = vol, no published port)    ▷ Cowrie's keys in the image
    ▷ the Docker daemon runs with userns-remap: root in c is unprivileged on the host
    db.add_decoy(c, status = starting)
    wait until C[svc].port accepts a connection or 5 s pass
    if not ready then RETIRE(c, status = failed) ; return ∅                          ▷ its logs are kept
    if svc = ssh then ADD-LOGINS(c, L)                              ▷ same user and password as in Cowrie
    db.set_status(c, running) ; return d ← (c.id, c.address, C[svc].port, π, ip)

procedure DECOY-MONITOR()                                                                 ▷ periodic
    for each running decoy d do
        if idle(d) > T_idle or age(d) > T_max then RETIRE(d)

procedure RETIRE(d, status = retired)                                    ▷ evidence first, then removal
    save the container log (with timestamps) and the container description of d
    if svc(d) = ssh then archive /home, /root and /tmp from inside d             ▷ what the attacker left
    write SHA256SUMS over all evidence files
    REDIRECTOR.remove(d.ip, d.port, d) ; stop and remove container d
    db.close_decoy(d, status, stop time)
```

Algorithm 2.15 installs and maintains the baseline, isolation and DNAT rules.

Algorithm 2.15: REDIRECTOR

File: redirector.py Phase: GP1 (planned)
Input : (attacker address ip, destination port, decoy d)
Output: Netfilter rules tagged with the comment acdf:<decoy id>

```
procedure install_baseline()
    ensure nat PREROUTING: -i ens33 -p tcp --dport 22 -j REDIRECT --to-ports 2222   ▷ acdf:cowrie-redirect
    ensure decoynet: bridge br-acdf, subnet 172.30.0.0/24, no masquerade               ▷ acdf-net service
    DOCKER-USER: return replies and decoy-to-decoy traffic;
        log and drop every other NEW connection from decoynet
    INPUT: accept replies; log and drop any other packet from decoynet to the host

procedure add(ip, port, d)
    r ← -t nat -I PREROUTING 1 -s ip -p tcp --dport port
        -j DNAT --to-destination d.addr:d.port -m comment --comment acdf:d.id
    if r is not present (iptables -C) then insert r                          ▷ above the REDIRECT rule
    db.add_rule(r)
    ▷ existing connections keep their conntrack entry and stay in Cowrie

procedure remove(ip, port, d)
    delete the rule with comment acdf:d.id if present ; db.close_rule(...)

procedure flush()
    delete every rule whose comment starts with acdf:
```

Algorithm 2.16 turns refused or unanswered connections to watched ports into probe events.

Algorithm 2.16: PROBE-WATCHER (self-morphing trigger)
File: probe_watcher.py Phase: GP1 (planned)
Input : Zeek conn.log (JSON), watched port set W, de-duplication window T_w = 60 s
Output: PROBE events (ip, port, time) placed on the orchestrator queue

```
follow conn.log as it grows (tail -F semantics, survives log rotation)
for each record c do
    if c.resp_h ≠ host address or c.resp_p ∉ W then continue
    if c.conn_state ∉ {REJ, S0} then continue                          ▷ port closed or unanswered
    key ← (c.orig_h, c.resp_p)
    if key was emitted less than T_w ago then continue
    Q.put(PROBE(ip = c.orig_h, port = c.resp_p, time = c.ts))
▷ the probe that triggers a decoy is refused; the attacker's next attempt
▷ reaches the decoy. An NFLOG rule can replace conn.log if latency is too high
```

Algorithm 2.17 changes the content of idle decoys without breaking open sessions; the host identity stays fixed.

Algorithm 2.17: MTD-ROTATE
File: mtd.py Phase: GP1 (planned)
Input : running decoys, profile pool per service, period P_MTD with random jitter
Output: decoys recreated with new content (host identity unchanged); rotation records

```
for each running decoy d do
    if d has open connections (conntrack) then continue                ▷ never break a live session
    if d is inside an active engagement (idle < T_idle) then continue
    π' ← draw a profile for d.svc with π' ≠ d.π :
        file set and lure credentials, accounts, watched-port mapping          ▷ never host identity
    d' ← DEPLOY-DECOY(d.ip, d.svc, π', logins(d.ip))                                ▷ Algorithm 2.14
    if d' = ∅ then continue                                                  ▷ keep the old decoy
    REDIRECTOR.add(d.ip, d.port, d') ; REDIRECTOR.remove(d.ip, d.port, d)
    RETIRE(d) ; db.log_rotation(d, d', π')
schedule the next run at now + P_MTD ± jitter
```

Algorithm 2.18 prepares and validates honey-files with a small language model before deployment.

Algorithm 2.18: GENERATE-CONTENT (honey-files)
File: content_gen.py Phase: GP1 (planned)
Input : service svc, profile π, templates T(svc), small instruction model M
(Qwen2.5-1.5B/3B-Instruct or Phi-3.5-mini, GGUF Q4, llama-cpp-python;
the model is chosen by a comparison defined in advance)
Output: directory of honey-files mounted into the decoy

```
key ← hash(svc, π.seed) ; if cache[key] exists then return cache[key]
▷ runs offline, before deployment: never on the attacker's path, so the
▷ generation latency cannot change the timing of an attacker's session
load M (one instance, unloaded after idle time)
for each template t in T(svc) do                          ▷ e.g. .bash_history, config.php, auth.log
    prompt ← t.instruction filled ONLY with profile fields (host, users, dates)
    ▷ attacker input never reaches the model (no prompt injection path)
    text ← model.generate(prompt, max_tokens = t.max, temperature = 0.7)
    if not VALID(text, t, π) then text ← t.static_fallback
    files[t.path] ← text
write files to a new volume ; cache[key] ← volume ; return volume

function VALID(text, t, π)
    return length within t.bounds and the format check passes (log / SQL / PHP)
        and every host, user and date matches π and no forbidden pattern appears
```

Algorithm 2.19 defines the SQLite store that records and replays every decision.

Algorithm 2.19: STATE-DB

```
File  : orchestrator/state_db.py + schema.sql (SQLite)
Phase : GP1 (implemented); integration planned
Input : records from the orchestrator
Output: auditable, replayable history of every decision
```

```
 1  open database with WAL journal mode and foreign keys on
 2  create tables if missing:
 3      sessions (session_id, src_ip, username, started, ended, n_lines, verdict,
 4                p_human, model_name, features_json)                  ▷ p_human = score p of Algorithm 2.12
 5      decisions(id, ts, event_type, src_ip, session_id, action, reason_json, latency_ms)
 6      decoys   (decoy_id, svc, profile_json, container_id, addr, port, attacker_ip,
 7                started, stopped, status)
 8      rules    (rule_id, decoy_id, src_ip, dport, target, comment, added, removed)
 9      scans    (scan_id, ts, services_json)
10  procedure log_decision(ev, ctx, verdict, p, action)
11      within one transaction: upsert sessions row; insert decisions row
12  procedure replay(t1, t2)
13      return decisions between t1 and t2 joined with decoys and rules, ordered by ts
```